



Chiang Mai University

UndefinedAC + HomuraAkemi + Intercontinental Ballistic Missile CMU

Suthana Kwaonueng, Theerada Siri, Paisit Lertananpipat

2025-09-04

- 1 Contest
- 2 Tanya’s Library
- 3 cellul4r
- 4 Mathematics

Contest (1)

template.cpp80 lines

```
#include<bits/stdc++.h>
using namespace std;
void __print(int x) {cerr << x;}
void __print(long x) {cerr << x;}
void __print(long long x) {cerr << x;}
void __print(unsigned x) {cerr << x;}
void __print(unsigned long x) {cerr << x;}
void __print(unsigned long long x) {cerr << x;}
void __print(float x) {cerr << x;}
void __print(double x) {cerr << x;}
void __print(long double x) {cerr << x;}
void __print(char x) {cerr << '\\' << x << '\\';}
void __print(const char *x) {cerr << '\"' << x << '\"';}
void __print(const string &x) {cerr << '\"' << x << '\"';}
void __print(bool x) {cerr << (x ? "true" : "false");}

template<typename T, typename V>
void __print(const pair<T, V> &x);
template<typename T>
void __print(const T &x) {int f = 0; cerr << '{'; for (auto &i:
x) cerr << (f++ ? ", " : ""), __print(i); cerr << "}";}
template<typename T, typename V>
void __print(const pair<T, V> &x) {cerr << '{'; __print(x.first
); cerr << ", "; __print(x.second); cerr << '}';}
void _print() {cerr << "]\n";}
template <typename T, typename... V>
void _print(T t, V... v) {__print(t); if (sizeof...(v)) cerr <<
", "; _print(v...);}

//#ifdef DEBUG
#define dbg(x...) cerr << "\e[91m"<<__func__<<":"<<__LINE__<<"
[" << #x << "]" = ["; _print(x); cerr << "\e[39m" << endl;
//#else
//#define dbg(x...)
//endif

typedef long long ll;
typedef long double ld;
typedef complex<ld> cd;

typedef pair<int, int> pi;
typedef pair<ll,ll> pl;
typedef pair<ld,ld> pd;

typedef vector<int> vi;
typedef vector<ld> vd;
typedef vector<ll> vl;
typedef vector<pi> vpi;
typedef vector<pl> vpl;
typedef vector<cd> vcd;
```

```
//template<class T> using pq = priority_queue<T>;
//template<class T> using pqg = priority_queue<T, vector<T>,
greater<T>>;
```

111516

```
#define rep(i, a) for(int i=0;i<a;++i)
#define FOR(i, a, b) for (int i=a; i<(b); i++)
#define F0R(i, a) for (int i=0; i<(a); i++)
#define FORd(i,a,b) for (int i = (b)-1; i >= a; i--)
#define F0Rd(i,a) for (int i = (a)-1; i >= 0; i--)
#define trav(a,x) for (auto& a : x)
#define uid(a, b) uniform_int_distribution<long long>(a, b)(rng
)

#define sz(x) (int)(x).size()
#define mp make_pair
#define pb push_back
//define f first
//define s second
#define lb lower_bound
#define ub upper_bound
#define all(x) x.begin(), x.end()
#define ins insert

template<class T> bool ckmin(T& a, const T& b) { return b < a ?
a = b, 1 : 0; }
template<class T> bool ckmax(T& a, const T& b) { return a < b ?
a = b, 1 : 0; }

mt19937 rng(chrono::steady_clock::now().time_since_epoch().
count());

const char nl = '\n';
const int N = 2e5+3;
const int INF = INT_MAX;
const long long LINF = LLONG_MAX;

int main(){
ios::sync_with_stdio(false);cin.tie(nullptr);
}

.vimrc21 lines

"General editor settings
set tabstop=4
set nocompatible
set shiftwidth=4
set expandtab
set autoindent
set smartindent
set ruler
set showcmd
set incsearch
set shellslash
set number
set relativenumber
set cino+=L0

"keybindings for { completion, "jk" for escape, ctrl-a to
select all
inoremap {<CR> {<CR><Esc>O
inoremap {} {}
imap jk <Esc>
map <C-a> <esc>ggVG<CR>
set belloff=all

.bashrc1 lines

export PATH=$PATH:~/scripts/

build.sh1 lines

g++ -static -DLOCAL -lm -s -x c++ -Wall -Wextra -O2 -std=c++17
-o $1 $1.cpp
```

stress.sh22 lines

```
#!/usr/bin/env bash

for ((testNum=0;testNum<$4;testNum++))
do
./$3 > input
./$2 < input > outSlow
./$1 < input > outWrong
H1=`md5sum outWrong`
H2=`md5sum outSlow`
if !(cmp -s "outWrong" "outSlow")
then
echo "Error found!"
echo "Input:"
cat input
echo "Wrong Output:"
cat outWrong
echo "Slow Output:"
cat outSlow
exit
fi
done
echo Passed $4 tests
```

Tanya’s Library (2)

2.1 Template & Utilities

pragma.h1021ce, 7 lines

Description: random pragma magic that I don’t fucking know but occasionaly solve my stupid bad implementation code

```
//some one on cf told us that these are mostly enough
#pragma GCC target ("avx2")
#pragma GCC optimize ("O3")
#pragma GCC optimize ("unroll-loops")
//just in case have this one
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target ("avx2,bmi,bmi2,popcnt,lzcnt")
```

2.2 Basic & Combinatorics

binpow.hd4debd, 11 lines

Description: TODO

```
long long binpow(long long a, int n){
long long res = 1;
while(n>0){
if(n&1){
res = res * a;
}
a = a * a;
n>>=1;
}
return res;
}

bigmod.h91cb0a, 11 lines

Description: TODO



```
long long bigmod(long long a, int n){
long long res = 1;
while(n>0){
if(n&1){
res = res * a % MOD;
}
a = a * a % MOD;
n>>=1;
}
```


```

```
    }
    return res;
}
```

binomialCoefficients.h

Description: TODO

3f7261, 10 lines

```
//more practical to precal C[n][r] with pascal triangle...
//...if constrain is O(n^2) able
long long C(int n, int k) {
    if(k==0)return 1;
    else if(k<0 || n<k)return 0;
    double res = 1;
    for (int i = 1; i <= k; ++i)
        res = res * (n - k + i) / i;
    return (long long) (res + 0.01);
}
```

matrix.h

Description: Nothing to says

5da844, 33 lines

```
struct Matrix {
    vector<vector<long long>>> a;
    int rows, cols;

    Matrix(int r, int c, int val) : rows(r), cols(c), a(r,
        vector<long long>(c, val)) {}

    Matrix operator *(const Matrix& other) {
        Matrix product(rows, other.cols, 0);

        for (int i = 0; i < rows; ++i) {
            for (int j = 0; j < other.cols; ++j) {
                for (int k = 0; k < cols; ++k) {
                    product.a[i][j] += a[i][k] * other.a[k][j]
                        % INF;
                    product.a[i][j] %= INF;
                }
            }
        }
        return product;
    }
};

Matrix expo_power(Matrix a, long long n, int size){//note n=1
    is equal do nothing,n=2is do once
    Matrix product(size,size,0);
    for(int i=0;i<size;++i)product.a[i][i]=1;
    while(n>0){
        if(n&1){
            product = product * a;
        }
        a = a * a;
        n>>=1;
    }
    return product;
}
```

mobius-function.h

Description: TODO

c6a8f4, 10 lines

```
int mob[N+1];
void mobius() {
    mob[1] = 1;
    for (int i = 2; i <= N; i++){
        mob[i]--;
        for (int j = i + i; j <= N; j += i) {
            mob[j] -= mob[i];
        }
    }
}
```

div-ceil-floor.h

Description: to stop c++ fucked up negative case of ceil, floor

2834e8, 9 lines

```
long long div_floor(__int128 a, long long b){
    __int128 q = a / b;
    __int128 r = a % b;
    if (r != 0 && ((a < 0) != (b < 0))) --q;
    return (long long)q;
}

long long div_ceil(__int128 a, long long b){
    return -div_floor(-a, b);
}
```

factoradix.h

Description: TODO

e835c3, 63 lines

```
<ext/pb_ds/assoc.container.hpp>, <ext/pb_ds/tree_policy.hpp>
typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>,
    __gnu_pbds::rb_tree_tag, __gnu_pbds::
        tree_order_statistics_node_update> ordered_set;

ordered_set st;
//st.order_of_key(x) - find # of elements in st stricly less
    than x
//st.size() - size of st
//st.find_by_order(x) - return iterator to the x-th largest
    element
//st.clear() - clear container

vector<int> factoradix_to_permutation(int n, vector<int> &
    factoradix){
    st.clear(); for(int i=1;i<=n;++i) st.ins(i);
    int cl = 0;
    while((int)factoradix.size() < n){
        factoradix.pb(0); ++cl;
    }

    vector<int> res;
    for(int i=(int)factoradix.size()-1;i>=0;--i){
        auto it = st.find_by_order(factoradix[i]);
        res.pb(*it);
        st.erase(it);
    }

    while(cl-->0) factoradix.pop_back();

    return res;
}

vector<int> permutation_to_factoradix(int n, vector<int> &
    permutation){
    st.clear(); for(int i=1;i<=n;++i) st.ins(i);
    vector<int> res;
    for(int i=0;i<(int)permutation.size();++i){
        int cnt = st.order_of_key(permutation[i]);
        res.pb(cnt);
        auto it = st.find_by_order(cnt); st.erase(it);
    }
    reverse(all(res));
    while(!res.empty() and res.back() == 0)res.pop_back();

    return res;
}

vector<int> decimal_to_factoradix(ll n){
    vector<int> res;
    int d = 1;
```

```
    while(n > 0){
        res.pb(n % d);
        n/=d; ++d;
    }
    return res;
}

ll factoradix_to_decimal(vector<int> &cur){
    ll res = 0;
    ll d = 1;
    for(int i=0;i<(int)cur.size();++i){
        res += d*cur[i];
        d*=(i+1);
    }
    return res;
}
```

//TODO using 0 base indexing, i.e. your 1st lexicographically permutation now is start at 0th

gauss-jordan-elimination.h

Description: TODO

d847fe, 48 lines

```
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be infinity
    or a big number

int gauss (vector < vector<double> > a, vector<double> & ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;

    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;

        if (abs (a[sel][col]) < EPS)
            continue;

        for (int i=col; i<=m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }

    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i];

    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0; // contradiction means no solution
    }

    for (int i=0; i<m; ++i)
        if (where[i] == -1)
```

```
        return INF; //infinite solution
    return 1; //unique solution
}
```

2.3 Primes & Factorization

sieve.h

Description: TODO

abb3a3, 22 lines

```
//construct is O(sqrt(n))
//can use to find all prime factor of a number in O(log n)
const int M = 2e5+1;
vector<bool> is_prime(M+1, true);
void sieve(){
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; i * i <= M; i++) {
        if (is_prime[i]) {
            for (int j = i * i; j <= M; j += i)
                is_prime[j] = false;
        }
    }
    /* log n sieve (use sieve to find all prime factors in O(
        log n) )
    for (int i = 2; i <= M; i++)is_prime[i] = i;
    for (int i = 2; i * i <= M; i++) {
        if (is_prime[i] == i) {
            for (int j = i * i; j <= M; j += i)
                is_prime[j] = i;
        }
    }
    */
}
```

linear-sieve.h

Description: TODO

d893f1, 18 lines

```
const int N = 10000000;
vector<int> lp(N+1); //lp[i] = minimum prime that divide i
vector<int> pr;

void sieve(){
    for (int i=2; i <= N; ++i) {
        if (lp[i] == 0) {
            lp[i] = i;
            pr.push_back(i);
        }
        for (int j = 0; i * pr[j] <= N; ++j) {
            lp[i * pr[j]] = pr[j];
            if (pr[j] == lp[i]) {
                break;
            }
        }
    }
}
```

phi-sieve.h

Description: TODO

b4e4ad, 15 lines

```
//set value for M
const int M = 1e6;
vector<int> phi(M+1);

void phi_1_to_n() {
    for (int i = 0; i <= M; i++)
        phi[i] = i;

    for (int i = 2; i <= M; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= M; j += i)
                phi[j] -= phi[j] / i;
```

```
        }
    }
}
```

phi-function.h

Description: TODO

fb8d57, 14 lines

```
//O(sqrt n)
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}
```

mobius-function.h

Description: TODO

c6a8f4, 10 lines

```
int mob[N+1];
void mobius() {
    mob[1] = 1;
    for (int i = 2; i <= N; i++){
        mob[i]--;
        for (int j = i + i; j <= N; j += i) {
            mob[j] -= mob[i];
        }
    }
}
```

miller-rabin.h

Description: TODO

980e40, 52 lines

```
//randomize primality test O(logn^2)
using u64 = uint64_t;
using u128 = __uint128_t;

u64 bigmod(u64 base, u64 e, u64 mod){
    u64 result = 1;
    base %= mod;
    while(e){
        if(e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}

bool check_composite(u64 n, u64 a, u64 d, int s){
    u64 x = bigmod(a, d, n);
    if(x == 1 || x == n - 1)
        return false;
    for(int r = 1; r < s; r++){
        x = (u128)x * x % n;
        if(x == n - 1)
            return false;
    }
    return true;
}

//random number generator temporality disable because my
template already has it
```

```
//mt19937 rng(chrono::steady_clock::now().time_since_epoch().
count());
long long rnd(long long x, long long y) {
    return uniform_int_distribution<long long>(x, y)(rng);
}

bool MillerRabin(u64 n, int iter=5){ // return true if n is
probably prime, else return false.
    if(n < 4){
        return n == 2 || n == 3;
    }
    int s = 0;
    u64 d = n-1;
    while((d & 1) == 0){
        d >>= 1;
        s++;
    }

    for(int i = 0; i < iter; i++){
        long long a = rnd(2, n-2);
        if(check_composite(n, a, d, s))
            return false;
    }
    return true;
}
```

pollard-rho.h

Description: TODO

bc7c06, 20 lines

```
//randomize factorization in O(n^1/4 * logn)
long long mult(long long a, long long b, long long mod) {
    return (__int128)a * b % mod;
}

long long f(long long x, long long c, long long mod) {
    return (mult(x, x, mod) + c) % mod;
}

long long rho(long long n, long long x0=2, long long c=1) {
    long long x = x0, y = x0, g = 1;
    while (g == 1) {
        x = f(x, c, n);
        y = f(f(y, c, n), c, n);
        g = __gcd(abs(x - y), n);
    }

    if(g == n) return rho(n, uid(2, n-1), uid(-2, 2)); //uid is
uniform long
    else return g;
}
```

discrete-log.h

Description: TODO

f7650d, 33 lines

```
// Returns minimum x for which a ^ x % m = b % m.
int solve(int a, int b, int m) {
    a %= m, b %= m;
    int k = 1, add = 0, g;
    while ((g = __gcd(a, m)) > 1) {
        if (b == k)
            return add;
        if (b % g)
            return -1;
        b /= g, m /= g, ++add;
        k = (k * 111 * a / g) % m;
    }

    int n = sqrt(m) + 1;
    int an = 1;
```

```

    for (int i = 0; i < n; ++i)
        an = (an * 111 * a) % m;

    map<int, int> vals;
    for (int q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 111 * a) % m;
    }

    for (int p = 1, cur = k; p <= n; ++p) {
        cur = (cur * 111 * an) % m;
        if (vals.count(cur)) {
            int ans = n * p - vals[cur] + add;
            return ans;
        }
    }
    return -1;
}

```

primitive-root.h

Description: TODO

c6d472, 34 lines

```

/*
    The following code assumes that the modulo p is a prime
    number.
    To make it works for any value of p, we must add
    calculation of phi(p)
*/
int powmod (int a, int b, int p) {
    int res = 1;
    while (b)
        if (b & 1)
            res = int (res * 111 * a % p), --b;
        else
            a = int (a * 111 * a % p), b >=>= 1;
    return res;
}

int generator (int p) {
    vector<int> fact;
    int phi = p-1, n = phi; // cal phi p by assuming it's
        prime
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            fact.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        fact.push_back (n);

    for (int res=2; res<=p; ++res) {
        bool ok = true;
        for (size_t i=0; i<fact.size() && ok; ++i)
            ok &= powmod (res, phi / fact[i], p) != 1;
        if (ok) return res;
    }
    return -1;
}

```

crt.h

Description: TODO

efd085, 51 lines

```

class CRT { // ChineseRemainderTheorem
    typedef long long vlong;
    typedef pair<vlong,vlong> pll;
    vector<pll> equations;

```

public:

```

    void clear() {
        equations.clear();
    }
    void addEquation( vlong r, vlong m ) {
        equations.push_back({r, m});
    }

    void ext_gcd(vlong a, vlong b, vlong& x, vlong& y) {
        if (b == 0) {
            x = 1;
            y = 0;
            return;
        }
        vlong x1, y1;
        ext_gcd(b, a % b, x1, y1);
        x = y1;
        y = x1 - y1 * (a / b);
    }

    pll solve() {
        if (equations.size() == 0) return {-1,-1};

        vlong a1 = equations[0].first;
        vlong m1 = equations[0].second;
        a1 %= m1;
        for ( int i = 1; i < (int)equations.size(); i++ ) {
            vlong a2 = equations[i].first;
            vlong m2 = equations[i].second;

            vlong g = __gcd(m1, m2);
            if ( a1 % g != a2 % g ) return {-1,-1};
            vlong p, q;
            ext_gcd(m1/g, m2/g, p, q);

            vlong mod = m1 / g * m2;
            p = (p%mod+mod)%mod, q = (q%mod+mod)%mod;
            vlong x = ( (__int128)a1 * (m2/g) % mod *q % mod +
                (__int128)a2 * (m1/g) % mod * p % mod ) % mod;
            a1 = x;
            if ( a1 < 0 ) a1 += mod;
            m1 = mod;
        }
        return {a1, m1};
    }
};

```

diophantine.h

Description: TODO

4fcad2, 89 lines

```

ll gcd(ll a, ll b, ll& x, ll& y) { //gcd extended
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(ll a, ll b, ll c, ll &x0, ll &y0, ll &g)
{
    g = gcd(abs(a), abs(b), x0, y0);
    if (c % g) {
        return false;
    }

```

```

        x0 *= c / g;
        y0 *= c / g;
        if (a < 0) x0 = -x0;
        if (b < 0) y0 = -y0;
        return true;
    }

    void shift_solution(ll &x, ll &y, ll a, ll b, ll cnt) { //
        make sure that a,b coprime
        x += cnt * b;
        y -= cnt * a;
    }

    ll find_all_solutions(ll a, ll b, ll c, ll minx, ll maxx, ll
        miny, ll maxy) {
        ll x, y, g;
        if (!find_any_solution(a, b, c, x, y, g))
            return 0;
        a /= g;
        b /= g;

        ll sign_a = a > 0 ? +1 : -1;
        ll sign_b = b > 0 ? +1 : -1;

        shift_solution(x, y, a, b, (minx - x) / b);
        if (x < minx)
            shift_solution(x, y, a, b, sign_b);
        if (x > maxx)
            return 0;
        ll lx1 = x;

        shift_solution(x, y, a, b, (maxx - x) / b);
        if (x > maxx)
            shift_solution(x, y, a, b, -sign_b);
        ll rx1 = x;

        shift_solution(x, y, a, b, -(miny - y) / a);
        if (y < miny)
            shift_solution(x, y, a, b, -sign_a);
        if (y > maxy)
            return 0;
        ll lx2 = x;

        shift_solution(x, y, a, b, -(maxy - y) / a);
        if (y > maxy)
            shift_solution(x, y, a, b, sign_a);
        ll rx2 = x;

        if (lx2 > rx2)
            swap(lx2, rx2);

        ll lx = max(lx1, lx2);
        ll rx = min(rx1, rx2);
        if (lx > rx)
            return 0;

        return (rx - lx) / abs(b) + 1;
    }
}

//number of solution of ax + by = c in general
ll number_of_solution(ll a, ll b, ll c, ll x1, ll x2, ll y1, ll
    y2){
    if(a == 0 and b == 0 and c == 0)return (x2-x1+1)*(y2-y1+1);
    else if(a == 0 and b == 0) return 0;
    else if(a == 0){
        if(c%b!=0 or y1>c/b or y2<c/b)return 0;
        else return (x2-x1+1);
    } else if(b == 0){
        if(c%a!=0 or x1>c/a or x2<c/a)return 0;
        else return (y2-y1+1);
    }
}

```

```

    } else {
        return find_all_solutions(a, b, c, x1, x2, y1, y2);
    }
}

```

2.4 Arrays & Trees

dsu.h

Description: TODO

7efb0c, 23 lines

//TODO: initialized parent[] and _rank[] array

```

int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}

```

```

void make_set(int v) {
    parent[v] = v;
    _rank[v] = 1;
}

```

```

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (_rank[a] < _rank[b])
            swap(a, b);
        parent[b] = a;
        _rank[a] += _rank[b];
    }
}

```

fenwick-tree.h

Description: TODO

a9def9, 27 lines

//TODO: use 1 base indexing only!!! becareful
vector<long long>bit;

```

//range sum point update(k=new_val-old_val)
void add(int i, int k){//add k to ith data and it's parent
    range so on
    while(i<(int)bit.size()){
        bit[i] += k;
        i += i & -i; //add last set bit
    }
}

```

```

ll sum(int i){//culmulative sum to ith data
    ll sum=0;
    while(i>0){
        sum += bit[i];
        i -= i & -i;
    }
    return sum;
}

```

```

void build_bit(vl &a){
    bit=a;
    for(int i=1;i<(int)bit.size();++i){
        int p=i+(i&-i);//index to parent
        if(p<(int)bit.size())bit[p] += bit[i];
    }
}

```

normalSegtree.h

Description: TODO

d1ac68, 30 lines

```

//TODO: use 0 base indexing, initialize tree[] array (
//preferably 4 times size of nodes array
vector<long long>tree;
void update(int node,int n_l,int n_r,int q_i,long long value){
    if(n_r<q_i || q_i<n_l)return;
    if(q_i==n_l && n_r==q_i){
        tree[node] = value;
        return;
    }
    int mid = (n_r+n_l)/2;
    update(2*node,n_l,mid,q_i,value);
    update(2*node+1,mid+1,n_r,q_i,value);
    tree[node] = tree[2*node] + tree[2*node+1];
}

```

```

long long f(int node,int n_l,int n_r,int q_l,int q_r){
    if(n_r<q_l || q_r<n_l)return 0;
    if(q_l<=n_l && n_r<=q_r)return tree[node];
    int mid = (n_l+n_r)/2;
    return f(2*node,n_l,mid,q_l,q_r) + f(2*node+1,mid+1,n_r,q_l,q_r);
}

```

```

int build_tree(vi &a,int n){
    tree.clear();
    int m=n;
    while(__builtin_popcount(m)!=1)++m;
    tree.resize(2*m+10,0);
    for(int i=0;i<n;++i)tree[i+m]=a[i];
    for(int i=m-1;i>=1;--i)tree[i]=tree[2*i]+tree[2*i+1];
    return m;
}

```

lazySegtree.h

Description: TODO

59e123, 50 lines

```

//TODO: use 0 base indexing
vector<long long> tree,lazy;
void update(int node,int n_l,int n_r,int q_l,int q_r,int value)
{
    if(lazy[node]!=0){
        tree[node] += (long long)(n_r-n_l+1)*lazy[node]; // for
        range + update
        if(n_l!=n_r){
            lazy[2*node] += lazy[node];
            lazy[2*node+1] += lazy[node];
        }
        lazy[node] = 0;
    }
    if(n_r<q_l || q_r<n_l)return;
    if(q_l<=n_l && n_r<=q_r){
        tree[node] += (long long)(n_r-n_l+1)*value; // for range
        + update
        if(n_l!=n_r){
            lazy[2*node] += value;
            lazy[2*node+1] += value;
        }
        return;
    }
    int mid = (n_r+n_l)/2;
    update(2*node,n_l,mid,q_l,q_r,value);
    update(2*node+1,mid+1,n_r,q_l,q_r,value);
    tree[node] = tree[2*node] + tree[2*node+1];
}

```

```

long long f(int node,int n_l,int n_r,int q_l,int q_r){
    if(lazy[node]!=0){
        tree[node] += (long long)(n_r-n_l+1)*lazy[node];
        if(n_l!=n_r){

```

```

        lazy[2*node] += lazy[node];
        lazy[2*node+1] += lazy[node];
    }
    lazy[node] = 0;
}
if(n_r<q_l || q_r<n_l)return 0;
if(q_l<=n_l && n_r<=q_r)return tree[node];
int mid = (n_l+n_r)/2;
return f(2*node,n_l,mid,q_l,q_r) + f(2*node+1,mid+1,n_r,q_l,q_r);
}

int build_tree(vi &a,int n){
    tree.clear(); lazy.clear();
    int m=n;
    while(__builtin_popcount(m)!=1)++m;
    tree.resize(2*m+10,0); lazy.resize(2*m+10,0);
    for(int i=0;i<n;++i)tree[i+m]=a[i];
    for(int i=m-1;i>=1;--i)tree[i]=tree[2*i]+tree[2*i+1];
    return m;
}

```

persistent-segtree.h

Description: TODO

c4a26b, 38 lines

//Warning!This implementation is not garbage collected, so the
tree nodes aren't deleted even if the instance of the
segment tree is taken out of scope.

```

struct Node {
    ll val;
    Node *l, *r;

    Node(ll x) : val(x), l(nullptr), r(nullptr) {}
    Node(Node *ll, Node *rr) {
        l = ll, r = rr;
        val = 0;
        if (l) val += l->val;
        if (r) val += r->val;
    }
    Node(Node *cp) : val(cp->val), l(cp->l), r(cp->r) {}
};

```

```

int n, cnt = 1;
ll a[200001];
Node *roots[200001];

```

```

Node *build(int l = 1, int r = n) {
    if (l == r) return new Node(a[l]);
    int mid = (l + r) / 2;
    return new Node(build(l, mid), build(mid + 1, r));
}

```

```

Node *update(Node *node, int val, int pos, int l = 1, int r = n)
{
    if (l == r) return new Node(val);
    int mid = (l + r) / 2;
    if (pos > mid) return new Node(node->l, update(node->r, val, pos, mid + 1, r));
    else return new Node(update(node->l, val, pos, l, mid), node->r);
}

```

```

ll query(Node *node, int a, int b, int l = 1, int r = n) {
    if (l > b || r < a) return 0;
    if (l >= a && r <= b) return node->val;
    int mid = (l + r) / 2;
    return query(node->l, a, b, l, mid) + query(node->r, a, b, mid + 1, r);
}

```

merge-sort-tree.h

Description: TODO

<ext/pb_ds/assoc.container.hpp>, <ext/pb_ds/tree.policy.hpp>,
<ext/pb_ds/assoc.container.hpp>, <ext/pb_ds/tree.policy.hpp> 2d0612, 94 lines

//merge sort tree with segment tree

```
typedef __gnu_pbds::tree<pair<int,int>, __gnu_pbds::null_type,
    less<pair<int,int>>, __gnu_pbds::rb_tree_tag, __gnu_pbds::
    tree_order_statistics_node_update> ordered_set;
```

ordered_set st;

//st.order_of_key({x,-1}) - find # of elements in st stricly less than x

//st.clear() - clear container

vector<ordered_set> mtree;

```
ordered_set merge(ordered_set &a, ordered_set &b){
    ordered_set result;
    for(auto&p:a){
        result.ins(p);
    }
    for(auto&p:b){
        result.ins(p);
    }
    return result;
}
```

```
void update(int node,int n_l,int n_r,int q_i,int id,int old_val
,int value){
    if(n_r<q_i || q_i<n_l)return;
    if(q_i==n_l && n_r==q_i){
        auto it=mtree[node].find({old_val,id});
        mtree[node].erase(it);
        mtree[node].ins({value,id});
        return;
    }
    int mid = (n_r+n_l)/2;
    update(2*node,n_l,mid,q_i,id,old_val,value);
    update(2*node+1,mid+1,n_r,q_i,id,old_val,value);
    auto it=mtree[node].find({old_val,id});
    if(it!=mtree[node].end()){
        mtree[node].erase(it);
        mtree[node].ins({value,id});
    }
}
```

```
int f(int node,int n_l,int n_r,int q_l,int q_r, int value){
    if(n_r<q_l || q_r<n_l)return 0;
    if(q_l==n_l && n_r==q_r){
        return mtree[node].order_of_key({value,-1});
    }
    int mid = (n_l+n_r)/2;
    return f(2*node,n_l,mid,q_l,q_r,value)+f(2*node+1,mid+1,n_r
,q_l,q_r,value);
}
```

```
void build_mtree(vi &a){
    int n=(int)a.size();
    int m=n; while(__builtin_popcount(m)!=1)+m;
    //for(int i=0;i<2*m;++i)mtree[i].clear();
    mtree.resize(2*m);
    for(int i=0;i<n;++i)mtree[i+m].ins({a[i],i});
    for(int i=m-1;i>=1;--i)mtree[i]=merge(mtree[2*i],mtree[2*i
+1]);
}
```

//merge sort tree with fenwick tree(BIT) (4 times less space)

```
typedef __gnu_pbds::tree<pair<int,int>, __gnu_pbds::null_type,
    less<pair<int,int>>, __gnu_pbds::rb_tree_tag, __gnu_pbds::
    tree_order_statistics_node_update> ordered_set;
```

```
ordered_set st;
//st.order_of_key(x) - find # of elements in st stricly less
    than x
```

//TODO: use 1 base indexing

vector<ordered_set>bt;

```
void update(int i,int k,int old_value, int new_value){
    while(i<(int)bit.size()){
        auto it=bit[i].find({old_value,k});
        assert(it!=bit[i].end());
        if(it!=bit[i].end()){
            bit[i].erase(it);
        }
        bit[i].ins({new_value,k});
        i+=i&-i;//add last set bit
    }
}
```

int F(int i, int k){//culmulative sum to ith data

```
int sum=0;
while(i>0){
    sum+=bit[i].order_of_key({k,-1});
    i-=i&-i;
}
return sum;
}
```

```
void build_bit(vi &a){
    bit.resize((int)a.size());
    for(int i=1;i<(int)a.size();++i)bit[i].ins({a[i],i});
    for(int i=1;i<(int)bit.size();++i){
        int p=i+(i&-i);//index to parent
        if(p<(int)bit.size()){
            for(auto&x:bit[i])bit[p].ins(x);
        }
    }
}
```

2D-segtree.h

Description: TODO

ee2c74, 82 lines

```
// 2D segtree stored as tree[4*n][4*m]
vector<vector<ll>> tree;
int n, m; // This shit in global remember
vector<vector<ll>> A; // This shit in global too
```

```
void buildY(int vx, int lx, int rx, int vy, int ly, int ry) {
    if (ly == ry) {
        if (lx == rx) {
            tree[vx][vy] = A[lx][ly];
        } else {
            tree[vx][vy] = tree[vx*2][vy] + tree[vx*2+1][vy];
        }
    } else {
        int my = (ly + ry) >> 1;
        buildY(vx, lx, rx, vy*2, ly, my);
        buildY(vx, lx, rx, vy*2+1, my+1, ry);
        tree[vx][vy] = tree[vx][vy*2] + tree[vx][vy*2+1];
    }
}
```

```
void buildX(int vx = 1, int lx = 1, int rx = n) {
    if (lx != rx) {
        int mx = (lx+rx)>>1;
        buildX(vx*2, lx, mx);
        buildX(vx*2+1, mx+1, rx);
    }
    buildY(vx, lx, rx, 1, 1, m);
}
```

```
}

void updateY(int vx, int lx, int rx, int x,
    int vy, int ly, int ry, int y, ll val) {
    if (ly == ry) {
        if (lx == rx) {
            tree[vx][vy] = val;
        } else {
            tree[vx][vy] = tree[vx*2][vy] + tree[vx*2+1][vy];
        }
    } else {
        int my = (ly+ry)>>1;
        if (y <= my)
            updateY(vx,lx,rx,x, vy*2, ly, my, y, val);
        else
            updateY(vx,lx,rx,x, vy*2+1, my+1, ry, y, val);
        tree[vx][vy] = tree[vx][vy*2] + tree[vx][vy*2+1];
    }
}
```

```
void updateX(int vx, int lx, int rx, int x, int y, ll val) {
    if (lx != rx) {
        int mx = (lx+rx)>>1;
        if (x <= mx) updateX(vx*2, lx, mx, x, y, val);
        else
            updateX(vx*2+1, mx+1, rx, x, y, val);
    }
    updateY(vx, lx, rx, x, 1, 1, m, y, val);
}
```

```
ll queryY(int vx, int vy, int ly, int ry, int y1, int y2) {
    if (y2 < ly || ry < y1) return 0;
    if (y1 <= ly && ry <= y2) return tree[vx][vy];
    int my = (ly+ry)>>1;
    return queryY(vx,vy*2, ly, my, y1,y2)
        + queryY(vx,vy*2+1,my+1, ry, y1,y2);
}
```

```
ll queryX(int vx, int lx, int rx,
    int x1, int x2, int y1, int y2) {
    if (x2 < lx || rx < x1) return 0;
    if (x1 <= lx && rx <= x2)
        return queryY(vx, 1, 1, m, y1, y2);
    int mx = (lx+rx)>>1;
    return queryX(vx*2, lx, mx, x1,x2,y1,y2)
        + queryX(vx*2+1, mx+1, rx, x1,x2,y1,y2);
}
```

```
void build(int n, int m){
    tree.assign(4*(n+2), vector<ll>(4*(m+2), 0));
    buildX();
}
```

//using like this n,m = row,col dimention (1-base index)

//updateX(1,1,n,x,y,v);

//queryX(1,1,n,x1,x2,y1,y2)

order-statistics-tree.h

Description: TODO

<ext/pb_ds/assoc.container.hpp>, <ext/pb_ds/tree.policy.hpp> 033b41, 7 lines

```
typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>,
    __gnu_pbds::rb_tree_tag, __gnu_pbds::
    tree_order_statistics_node_update> ordered_set;
```

ordered_set st;

//st.order_of_key(x) - find # of elements in st stricly less than x

//st.size() - size of st

//st.find_by_order(x) - return iterator to the x-th largest element

```
//st.clear() - clear container
```

RMQ-1D.h

Description: TODO

```
e37d08, 25 lines
//general sparse table template(can be use for all other
    offline queries)
int rmq[N][20];

void build_rmq(vi &a){
    for(int j=0;j<20;++j){
        for(int i=0;i<(int)a.size();++i){
            if(j==0){
                rmq[i][0]=a[i];
            } else if(i+(1<=(j-1))<(int)a.size()){
                rmq[i][j]=min(rmq[i][j-1],rmq[i+(1<=(j-1))][j-1]);
            }
        }
    }
}

int query(int l, int r){
    int i=l,sub_array_size=r-l+1, ans=INF;
    for(int j=0;j<30;++j){
        if((1<=j)&(sub_array_size)){
            ans=min(ans,rmq[i][j]);
            i+=(1<=j);
        }
    }
    return ans;
}
```

RMQ-2D.h

Description: TODO

```
0f2f45, 46 lines
//general template for 2D offline queries
int rmq[N][N][20][20]; //N need to be around 1e2

void build_rmq(vector<vi>& a){
    for(int j=0;j<20;++j){
        for(int k=0;k<20;++k){
            for(int r=0;r<(int)a.size();++r){
                for(int c=0;c<(int)a[0].size();++c){
                    if(j==0&&k==0) rmq[r][c][0][0]=a[r][c];
                    else {
                        if(j==0 && k>0 && (c+(1<=(k-1))<(int)a[0].size() )){
                            rmq[r][c][j][k]=min(rmq[r][c][j][k-1],
                                rmq[r][c+(1<=(k-1))][j][k-1]);
                        } else if(k==0 && j>0 && (r+(1<=(j-1))<(int)a.size() )){
                            rmq[r][c][j][k]=min(rmq[r][c][j-1][k],
                                rmq[r+(1<=(j-1))][c][j-1][k]);
                        } else if( c+(1<=(k-1))<(int)a[0].size()
                            && r+(1<=(j-1))<(int)a.size() ){
                            rmq[r][c][j][k]=min({rmq[r][c][j-1][k-1],
                                rmq[r][c+(1<=(k-1))][j-1][k-1],
                                rmq[r+(1<=(j-1))][c][j-1][k-1],
                                rmq[r+(1<=(j-1))][c+(1<=(k-1))][j-1][k-1]});
                        }
                    }
                }
            }
        }
    }
}
```

```

    }
}

int query(int r1,int c1,int r2,int c2){
    int x=r1,y=c1,h=r2-r1+1,w=c2-c1+1,ans=INF;
    for(int j=0;j<30;++j){
        if((1<=j)&h){
            for(int k=0;k<30;++k){
                if((1<=k)&w){
                    ans=min(ans,rmq[x][y][j][k]);
                    y+=(1<=k);
                }
            }
        }
    }
    return ans;
}
```

2.5 Hashing & Tables

gp-hash-table.h

Description: TODO

```
3b295b, 7 lines
<ext/pb_ds/assoc.container.hpp>
using namespace __gnu_pbds;

const int RANDOM = chrono::high_resolution_clock::now().
    time_since_epoch().count();
struct chash {
    int operator()(int x) const { return x ^ RANDOM; }
};
gp_hash_table<int, int, chash> table;
```

cht.h

Description: TODO

```
87ad47, 28 lines
struct line {
    long long m, c;
    long long eval(long long x) { return m * x + c; }
    long double intersectX(line l) { return (long double) (c -
        l.c) / (l.m - m); }
};

deque<line> dq;
dq.push_front({0, 0});//cant be put in global, remove this to local function
//if query ask for minimum remove this line after 1st insertion

//TODO NOTE***: maximum and minimum value exist in bot left most and rightmost of convex hull so do search on both l to r and r to l

//constructing hull from l to r, maintain correct hull at rightmost
/* ***inserting line (maximum hull)
    line cur = line{...some m, ...some c}
    while(dq.size(>=2)&&cur.intersectX(dq.back())
        <=cur.intersectX(dq[dq.size()-2]))dq.pop_back();
    dq.pb(cur);
*/

//constructing hull from r to l, maintain correct hull at leftmost
/* inserting line (maximum hull)
    line cur = line{...some m, ...some c}
    while(dq.size(>=2)&&cur.intersectX(dq[0])>=cur.intersectX(
        dq[1])){
```

```

        dq.pop_front();
    }
    dq.push_front(cur);
*/

2.6 Graph Algorithms
dijkstra.h
Description: TODO
3fd08c, 17 lines
//initialize dist, proc
    for (int i = 1; i <= m; i++) dist[i] = INF;
    dist[x] = 0;
    priority_queue<pair<ll, int>>q;
    q.push({0,x});
    while (!q.empty()) {
        int a = q.top().second; q.pop();
        if (proc[a]) continue;
        proc[a] = true; // this fucker i dont remember all the time
        for (auto u : adj[a]) {
            int b = u.first, w = u.second;
            if (dist[a]+w < dist[b]) {
                dist[b] = dist[a]+w;
                q.push({-dist[b],b});
            }
        }
    }
```

floyd-warshall-stp.h

Description: before k-th phase $d[i][j]$ = stp len from i to j using only ver-tices from set 1,2,...k-1 as internal path, the beginning and end of path are not restricted by this property.

ab1def, 10 lines

```
//don't forget to initialize the base case of weight
//this work for negative weight without negative cycle
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}
```

kruskal-mst.h

Description: TODO

```
5dd25a, 37 lines
//kruskal
vector<tuple<int,int,int>>el;
int parent[N+1], _rank[N+1];

int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}

void make_set(int v) {
    parent[v] = v;
    _rank[v] = 1;
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (_rank[a] < _rank[b])
            swap(a, b);
        parent[b] = a;
    }
```



```
        _rank[a] += _rank[b];
    }
}

11 mst(int n){
    sort(all(el)); for(int i=1;i<=n;++i)make_set(i);
    11 mst_cost=0;
    for(auto&t:el){
        int w=get<0>(t), a=get<1>(t), b=get<2>(t);
        if(find_set(a) != find_set(b)){
            mst_cost+=w, union_sets(a,b);
        }
    }
    return mst_cost;
}
```

prim-mst.h
Description: TODO a70ceb, 24 lines

*/*note: unlike kruskal, prim's algorithm assume all vertices belong in same graph*
vector<pair<int,int>>adj[N+1];
bool taken[N+1];
priority_queue<pair<int,int>>PQ;

```
void process(int v){
    taken[v]=1;
    for(auto&p:adj[v]){
        int b=p.second, w=p.first;
        if(!taken[b])PQ.push({-w,-b});
    }
}
```

```
11 mst(){
    process(1);
    11 mst_cost=0;
    while(!PQ.empty()){
        pair<int,int> front=PQ.top(); PQ.pop();
        int u=-front.second, w=-front.first;
        if(!taken[u])
            mst_cost+=w, process(u);
    }
    return mst_cost;
}
```

dinic.h
Description: TODO ce6220, 82 lines

```
/*COPY FROM BENQ GITHUB I dont fucking know how this work
/*General graphs: O(V^2E);
/*Bipartite matching(unit caps): O(E sqrt2(V))
struct Dinic {
    using F = long long; // flow type
    struct Edge { int to; F flow, cap; };
    int N;
    vector<Edge> edges;
    vector<vector<int>> adj;
    vector<int> level;
    vector<vector<int>::iterator> it;

    // Initialize Dinic for a graph with _N vertices (0-indexed or 1-indexed)
    void init(int _N) {
        N = _N;
        adj.assign(N, {});
        edges.clear();
        level.resize(N);
        it.resize(N);
    }
}
```

```
// Add edge u -> v with capacity 'cap', and reverse edge v -> u with capacity 'rev_cap'
void addEdge(int u, int v, F cap, F rev_cap = 0) {
    // forward edge
    adj[u].push_back((int)edges.size());
    edges.push_back({v, 0, cap});
    // reverse edge
    adj[v].push_back((int)edges.size());
    edges.push_back({u, 0, rev_cap});
}
```

```
// Build level graph via BFS
bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    for (int i = 0; i < N; i++) it[i] = adj[i].begin();
    queue<int> q;
    level[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int idx : adj[u]) {
            const Edge &e = edges[idx];
            if (level[e.to] < 0 && e.flow < e.cap) {
                level[e.to] = level[u] + 1;
                q.push(e.to);
            }
        }
    }
    return level[t] >= 0;
}
```

```
// DFS to send flow
F dfs(int u, int t, F pushed) {
    if (u == t || pushed == 0) return pushed;
    for (; it[u] != adj[u].end(); ++it[u]) {
        int idx = *it[u];
        Edge &e = edges[idx];
        if (level[e.to] != level[u] + 1 || e.flow == e.cap)
            continue;
        F tr = dfs(e.to, t, min(pushed, e.cap - e.flow));
        if (tr > 0) {
            e.flow += tr;
            edges[idx ^ 1].flow -= tr; // reverse edge
            return tr;
        }
    }
    return 0;
}
```

```
// Compute max flow from s to t
F maxFlow(int s, int t) {
    F flow = 0;
    while (bfs(s, t)) {
        while (F pushed = dfs(s, t, numeric_limits<F>::max())) {
            flow += pushed;
        }
    }
    return flow;
}
```

// After maxFlow, vertices reachable from s in residual graph have level >= 0.
// Edges from reachable u to unreachable v form a min-cut.

};

eulerian-cycle.h
Description: TODO 87df80, 25 lines

*/*TODO: checking all node degrees is even ?*
*/*TODO: checking if all edges are connectedly reachable*

```
stack<int> st;
vi res;
st.push(1);
while(!st.empty()){
    int v = st.top();
    if(deg[v] == 0){
        res.pb(v);
        st.pop();
    } else {
        int rm = -1; //must exist
        for(auto &b:adj[v]){
            rm = b;
            break;
        }
        adj[v].erase(rm); deg[v]--;
        adj[rm].erase(v); deg[rm]--;
        st.push(rm);
    }
}
//reverse the res if the edge is directed !! also changing the checking condition
for(auto &x:res)cout << x << " ";
cout << nl;
```

2.7 Graph Decompositions & Connectivity

articulation-points.h
Description: TODO 23f413, 37 lines

int n; *// number of nodes*
vector<vector<int>> adj; *// adjacency list of graph*

vector<bool> visited;
vector<int> tin, low;
int timer;

```
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children=0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p!=-1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if(p == -1 && children > 1)
        IS_CUTPOINT(v);
}
```

```
void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
```

```
        if (!visited[i])
            dfs (i);
    }
}
```

bridge-finding.h

Description: TODO

747a19, 30 lines

```
//NOTE*: not work for multigraph, I'm too lazy too fix for now
/*
    dfs-tree bridge finding algorithm
    let lvl[root] = 1 (root = 1)
*/
int root = 1;
vector<pair<int,int>> bridge;
int dp[N+1], lvl[N+1];

void dfs_tree(int a, int e){
    if(a == root)lvl[a] = 1;
    dp[a] = 0;
    for(auto &b:adj[a]){
        if(b == e)continue;

        if(lvl[b] == 0){
            lvl[b] = lvl[a] + 1;
            dfs_tree(b, a);
            dp[a] += dp[b];
        } else if(lvl[b] < lvl[a]){
            dp[a]++;
        } else if(lvl[b] > lvl[a]){
            dp[a]--;
        }
    }

    if(lvl[a] > 1 and dp[a] == 0){
        bridge.pb({min(a,e), max(a,e)});
    }
}
```

tarjan-scc.h

Description: TODO

3e5381, 35 lines

```
//Trajan's algorithm
vi adj[N+1];
int id[N+1], low[N+1];
bool vst[N+1], in_stack[N+1];

stack<int>st;
vector<vi>scc;
vi comp;
int tin=0;
void dfs(int a){
    st.push(a);
    vst[a]=in_stack[a]=1;
    id[a]=low[a]=tin++;

    for(auto&b:adj[a]){
        if(!vst[b]){
            dfs(b);
            low[a]=min(low[a],low[b]); //Backtrack low-link value
        } else if(in_stack[b]){
            //Backtrack low-link value
            low[a]=min(low[a],low[b]);
        }
    }
    if(id[a] == low[a]){ //pop scc
        while(!st.empty()){
            int v=st.top();
```

```
        comp.pb(v), st.pop();
        in_stack[v]=0;
        if(v==a)break; //stop when scc is found
    }
}
if(!comp.empty()){
    scc.pb(comp), comp.clear();
}
}
```

centroid-decomposition.h

Description: TODO

70d410, 32 lines

```
vector<int> adj[N];
bool is_removed[N];
int subtree_size[N];
int get_subtree_size(int node, int parent = -1) {
    subtree_size[node] = 1;
    for (int child : adj[node]) {
        if (child == parent || is_removed[child]) { continue; }
        subtree_size[node] += get_subtree_size(child, node);
    }
    return subtree_size[node];
}
int get_centroid(int node, int tree_size, int parent = -1) {
    for (int child : adj[node]) {
        if (child == parent || is_removed[child]) { continue; }
        if (subtree_size[child] * 2 > tree_size) {
            return get_centroid(child, tree_size, node);
        }
    }
    return node;
}
void build_centroid_decomp(int node = 0) {
    int centroid = get_centroid(node, get_subtree_size(node));
    is_removed[centroid] = true;

    // do something

    for (int child : adj[centroid]) {
        if (is_removed[child]) { continue; }
        build_centroid_decomp(child);
    }
}
```

hld.h

Description: TODO

e4ffaf, 152 lines

```
/*hld*/
//TODO: initialize constant N
vector<int>adj[N];

void add_edges(int a, int b){
    adj[a].pb(b);
    adj[b].pb(a);
}

const int LOG = 20;
int timer = 0, seg_tree_size = 0;
int depth[N], parent[N], n_subtree[N], heavyChild[N], chain[N],
    SegTreePos[N];
int cost[N];
int up[N][LOG]; // 2^j-th ancestor of n

/*SegTree*/
//finding max edge query
ll tree[4*N];
void update(int node,int n_l,int n_r,int q_i,int value){
    if(n_r<q_i || q_i<n_l)return;
```

```
    if(q_i==n_l && n_r==q_i){
        tree[node] = value;
        return;
    }
    int mid = (n_r+n_l)/2;
    update(2*node,n_l,mid,q_i,value);
    update(2*node+1,mid+1,n_r,q_i,value);
    tree[node] = max(tree[2*node], tree[2*node+1]);
}

long long f(int node,int n_l,int n_r,int q_l,int q_r){
    if(n_r<q_l || q_r<n_l)return 0;
    if(q_l<=n_l && n_r<=q_r)return tree[node];
    int mid = (n_l+n_r)/2;
    return max(f(2*node,n_l,mid,q_l,q_r) , f(2*node+1,mid+1,n_r
        ,q_l,q_r));
}
/*SegTree*/

//preprocessing the arrays above
void dfs(int a,int e){
    chain[a] = a;
    n_subtree[a] = 1;
    int bestSon = -1, hmax = -1;
    for(auto b:adj[a]){
        if(b == e)continue;
        depth[b] = depth[a] + 1;
        parent[b] = a;
        up[b][0] = parent[b];
        for(int i=1;i<LOG;++i){
            up[b][i] = up[up[b][i-1]][i-1];
        }
        dfs(b,a);
        n_subtree[a] += n_subtree[b];
        if(n_subtree[b] > hmax){
            hmax = n_subtree[b], bestSon = b;
        }
    }
    heavyChild[a] = bestSon;
}

void computeHeavyChains(int a, int e){
    if(heavyChild[a] != -1){
        chain[heavyChild[a]] = chain[a];
    }
    for(auto b:adj[a]){
        if(b == e)continue;
        computeHeavyChains(b, a);
    }
}

void setHeavyChains(int a, int e){
    SegTreePos[a] = timer++;
    if(heavyChild[a] != -1){
        setHeavyChains(heavyChild[a], a);
        int w = 0;
        for(auto b:adj[a]){
            if(b == heavyChild[a]){
                w = 1;
                break;
            }
        }
        //setting cost of heavyChild[a] = weight of edge (a,
            heavyChild[a])
        cost[heavyChild[a]] = w; //change when tree is weighted
    }
    for(auto b:adj[a]){
        if(b == e || heavyChild[a] == b)continue;
        cost[b] = 1; //change when tree is weighted
```

```

        setHeavyChains(b , a);
    }
}

void buildSegTree(int n_node){
    seg_tree_size = n_node;
    while(__builtin_popcount(seg_tree_size)!=1)++seg_tree_size;
    for(int i=0;i<n_node;++i){
        update(1,0,seg_tree_size-1,SegTreePos[i], cost[i]);
    }
    for(int i=seg_tree_size-1;i>=1;--i){
        tree[i] = max(tree[2*i], tree[2*i+1]);
    }
}

ll queryPath(int mother, int son){
    ll ans = LONG_MIN;
    while(chain[mother] != chain[son]){
        if(chain[son] == son){
            ans = max(ans, f(1,0,seg_tree_size-1,SegTreePos[son]
                ],SegTreePos[son]));
        } else {
            ans = max(ans, f(1,0,seg_tree_size-1,SegTreePos[
                chain[son]],SegTreePos[son]));
        }
        son = parent[chain[son]];
    }

    ans = max(ans, f(1,0,seg_tree_size-1,SegTreePos[mother]+1,
        SegTreePos[son]));

    return ans;
}

int lca(int a, int b){
    if(depth[a]<depth[b])swap(a,b);
    int k = depth[a] - depth[b];
    for(int i=LOG-1;i>=0;--i){
        if(k & (1<<i)){
            a = up[a][i];
        }
    }

    if(a == b)return a;
    for(int i=LOG-1;i>=0;--i){
        if(up[a][i] != up[b][i]){
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}

ll query(int x, int y){
    int l = lca(x,y);
    return max( queryPath(l, x), queryPath(l, y)); //combine
        the answer
}

void hld_init(int n){
    dfs(0,-1);
    computeHeavyChains(0,-1);
    setHeavyChains(0,-1);
    buildSegTree(n);
}

/*hld*/

```

lca.h

Description: TODO

0ae36f, 36 lines

```

//TODO: initialize tree(adj list)
const int LOG = 20;
int dep[N], parent[N];
int up[N][LOG]; // 2^j-th ancestor of n

void dfs(int a,int e){
    for(auto b:adj[a]){
        if(b == e)continue;
        dep[b] = dep[a] + 1;
        parent[b] = a;
        up[b][0] = parent[b];
        for(int i=1;i<LOG;++i){
            up[b][i] = up[up[b][i-1]][i-1];
        }
        dfs(b,a);
    }
}

int lca(int a, int b){
    if(dep[a]<dep[b])swap(a,b);
    int k = dep[a] - dep[b];
    for(int i=LOG-1;i>=0;--i){
        if(k & (1<<i)){
            a = up[a][i];
        }
    }

    if(a == b)return a;
    for(int i=LOG-1;i>=0;--i){
        if(up[a][i] != up[b][i]){
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}

```

kth-ancestor.h

Description: TODO

3b29e6, 9 lines

```

int kth_ancestor(int node,int k){
    if(dep[node] < k)return -1;
    for(int i = 0;i < LOG; ++i){
        if(k & (1<<i)){
            node = up[node][i];
        }
    }
    return node;
}

```

tree-hashing.h

Description: TODO

ce0886, 22 lines

```

/*
    very acurate rooted tree hashing
    don't forget to check tree invariant such as tree size and
    # of centroid
*/
map<vector<int>, int> hasher;

int hashify(vector<int> x) {
    sort(x.begin(), x.end());
    if(!hasher[x]) {
        hasher[x] = hasher.size();
    }
    return hasher[x];
}

```

```

int _hash(int v, int e, vector<vector<int>>&adj) { // get a "
    hash" of v's subtree, e is parent vertex
    vector<int> children;
    for(int u: adj[v]) {
        if( u == e )continue;
        children.push_back(_hash(u, v, adj));
    }
    return hashify(children);
}

```

2.8 String Processing

AC-automaton.h

Description: TODO

a41621, 53 lines

```

const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;
    int p = -1;
    char pch;
    int link = -1;
    int go[K];

    Vertex(int p=-1, char ch='$') : p(p), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};

vector<Vertex> t(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].output = true;
}

int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0)
            t[v].link = 0;
        else
            t[v].link = go(get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch - 'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1)
            t[v].go[c] = t[v].next[c];
        else
            t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
    }
    return t[v].go[c];
}

```

kmp.h	
Description: TODO	4dfee5, 37 lines
<pre>int b[N]; int cnt = 0; void knp_proc(string t,string p){ int i=0,j=-1;b[0] = -1; while(i<(int)p.length()){ while(j>=0 && p[i]!=p[j]) j = b[j]; ++i; ++j; b[i] = j; } } void knp_search(string t,string p){<i>//count number of occurence of p in t</i> int i=0,j=0; while(i<(int)t.length()){ while(j>=0 && t[i] != p[j]) j = b[j]; ++i; ++j; if(j==(int)p.length()){ ++cnt; j = b[j]; } } } vector<int> prefix_function(string s) { int n = (int)s.length(); vector<int> pi(n); for (int i = 1; i < n; i++) { int j = pi[i-1]; while (j > 0 && s[i] != s[j]) j = pi[j-1]; if (s[i] == s[j]) j++; pi[i] = j; } return pi; }</pre>	

z-function.h	
Description: TODO	9a2512, 18 lines
<pre>vector<int> z_function(string s) { int n = s.size(); vector<int> z(n); int l = 0, r = 0; for(int i = 1; i < n; i++) { if(i < r) { z[i] = min(r - i, z[i - l]); } while(i + z[i] < n && s[z[i]] == s[i + z[i]]) { z[i]++; } if(i + z[i] > r) { l = i; r = i + z[i]; } } return z; }</pre>	

manacher.h	
Description: TODO	3069fe, 38 lines
<pre><i>//sub-palindrome queries</i> vector<int> manacher_odd(string s) { int n = s.size();</pre>	

	<pre>s = "\$" + s + "^"; vector<int> p(n + 2); int l = 1, r = 1; for(int i = 1; i <= n; i++) { p[i] = max(0, min(r - i, p[l + (r - i)])); while(s[i - p[i]] == s[i + p[i]]) { p[i]++; } if(i + p[i] > r) { l = i - p[i], r = i + p[i]; } } return vector<int>(begin(p) + 1, end(p) - 1); }</pre>
	<pre>vector<int> manacher(string s) { string t; for(auto c: s) { t += string("#") + c; } auto res = manacher_odd(t + "#"); return vector<int>(begin(res) + 1, end(res) - 1); }</pre>
	<pre><i>//beware the range is inclusive and therefore not half-open -> [l, r]</i> bool is_palindrome(int l, int r, vector<int> &v){ if(v[l + r] < (r - l + 1))return false; return true; }</pre>
	<pre>vector<int> build_manacher(string s){ <i>//0 base index string</i> auto v = manacher(s); for(auto&x:v)--x; return v; }</pre>

string-hashing.h	
Description: TODO	ba27df, 17 lines
<pre>typedef __int128 HASH; HASH p[NN], h[NN]; constexpr HASH M = 10000000000000000003; mt19937 rng(chrono::steady_clock::now().time_since_epoch(). count());<i>//delete this if using my template</i> #define uid(a, b) uniform_int_distribution<long long>(a, b)(rng) <i>//delete this if using my template</i> void compute_hash(string const& s){ p[0] = 1, p[1] = uid(256, M-1); for(ll i=0;i<(int)s.length();++i){ p[i+1] = p[i]*p[1]%M; h[i+1] = (h[i]*p[1] + s[i])%M; } } HASH sub_hash(ll l, ll r){ <i>//range half-open [l, r]</i> return (h[r] - p[r-l]*h[l]%M + M)%M; }</pre>	

double-hashing-string.h	
Description: TODO	276f01, 26 lines
<pre><i>//typedef</i> typedef long long ll; typedef pair<ll,ll> pl; #define M 1000000321 #define OP(x, y) pl operator x (pl a, pl b){return {a.first x b .first, (a.second y b.second) % M}; }</pre>	

	<pre>OP(+, +) OP(*, *) OP(-, + M -) <i>//random number generator</i> mt19937 gen(chrono::steady_clock::now().time_since_epoch(). count()); uniform_int_distribution<ll> dist(256, M - 1); <i>//queries - check if S[i:i+l] == S[j:j+l](inclusive), S is a string, [:] is slice</i> #define H(i, l) (h[(i) + (l)] - h[i] * p[l]) #define EQ(i, j, l) (H(i, l) == H(j, l)) <i>//preprocessing</i> const int N = 2e5; string s; pl p[N], h[N]; ll n; int main(){ cin >> n >> s; p[0] = {1,1}, p[1] = {dist(gen) 1, dist(gen) }; for(ll i=1;i<=(ll)s.length();++i){ p[i] = p[i-1] * p[1]; h[i] = h[i-1] * p[1] + make_pair(s[i-1], s[i-1]); } }</pre>
--	---

suffix-array.h	
Description: TODO	4ca004, 53 lines
<pre>vector<int> sort_cyclic_shifts(string const& s) { int n = s.size(); const int alphabet = 256; vector<int> p(n), c(n), cnt(max(alphabet, n), 0); for (int i = 0; i < n; i++) cnt[s[i]]++; for (int i = 1; i < alphabet; i++) cnt[i] += cnt[i-1]; for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i; c[p[0]] = 0; int classes = 1; for (int i = 1; i < n; i++) { if (s[p[i]] != s[p[i-1]]) classes++; c[p[i]] = classes - 1; } vector<int> pn(n), cn(n); for (int h = 0; (1 << h) < n; ++h) { for (int i = 0; i < n; i++) { pn[i] = p[i] - (1 << h); if (pn[i] < 0) pn[i] += n; } fill(cnt.begin(), cnt.begin() + classes, 0); for (int i = 0; i < n; i++) cnt[c[pn[i]]]++; for (int i = 1; i < classes; i++) cnt[i] += cnt[i-1]; for (int i = n-1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i]; cn[p[0]] = 0; classes = 1; for (int i = 1; i < n; i++) { pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]}; pair<int, int> prev = {c[p[i-1]], c[(p[i-1] + (1 << h)) % n]}; if (cur != prev) ++classes; } } }</pre>	

```
        cn[p[i]] = classes - 1;
    }
    c.swap(cn);
}
return p;
}

vector<int> suffix_array_construction(string s) {
    s += "$";
    vector<int> sorted_shifts = sort_cyclic_shifts(s);
    sorted_shifts.erase(sorted_shifts.begin());
    return sorted_shifts;
}
```

suffix-automaton.h

Description: TODO fe30a6, 52 lines

```
struct state {
    int len, link;
    map<char, int> next;
    map<char, int> go;
};

const int MAXLEN = 100001; //this one is 10^5 becareful
state st[MAXLEN * 2];
int sz, last;

void sa_init() {
    st[0].len = 0;
    st[0].link = -1;
    sz++;
    last = 0;
}

void sa_extend(char c) {
    int cur = sz++;
    st[cur].len = st[last].len + 1;
    int p = last;
    while (p != -1 && !st[p].next.count(c)) {
        st[p].next[c] = cur;
        p = st[p].link;
    }
    if (p == -1) {
        st[cur].link = 0;
    } else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = sz++;
            st[clone].len = st[p].len + 1;
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;
            while (p != -1 && st[p].next[c] == q) {
                st[p].next[c] = clone;
                p = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}

int sa_go(int v, char c) {
    if (st[v].go.count(c)) return st[v].go[c];
    if (st[v].next.count(c)) return st[v].go[c] = st[v].next[c];
    if (st[v].link == -1) return st[v].go[c] = 0; // fallback to root
}
```

suffix-automaton trie lyndon-factorization geometry

```
return st[v].go[c] = sa_go(st[v].link, c);
}

trie.h
Description: TODO e121b2, 26 lines

// string trie structure
const int K = 26;

struct node{
    int to[K];
    bool output = false;

    node(){
        fill(begin(to), end(to), -1);
    }
};

vector<node> trie(1);

void add(const& string s){
    int v = 0;
    for(char ch: s){
        int c = ch - 'a';
        if(trie[v][c] == -1){
            trie[v][c] = trie.size();
            trie.push_back(node());
        }
        v = trie[v][c];
    }
    trie[v].output = true;
}
```

lyndon-factorization.h

Description: TODO a3cc8c, 40 lines

```
vector<string> duval(string const& s) {
    int n = s.size();
    int i = 0;
    vector<string> factorization;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k) {
            factorization.push_back(s.substr(i, j - k));
            i += j - k;
        }
        return factorization;
    }
}

string min_cyclic_string(string s) {
    s += s;
    int n = s.size();
    int i = 0, ans = 0;
    while (i < n / 2) {
        ans = i;
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
        }
    }
}
```

```
        j++;
    }
    while (i <= k)
        i += j - k;
}
return s.substr(ans, n / 2);
}
```

2.9 Computational Geometry

geometry.h

Description: TODO 963fd3, 167 lines

```
//Geometry
#pragma GCC target("avx2")
#pragma GCC optimize("O3")
#pragma GCC optimize("unroll-loops")

typedef long long int ll;
typedef long double ld;
typedef complex<ld> pt;
struct line {
    pt P, D; bool S = false;
    line(pt p, pt q, bool b = false) : P(p), D(q - p), S(b) {}
    line(pt p, ld th) : P(p), D(polar((ld)1, th)) {}
};
struct circ { pt C; ld R; };

#define X real()
#define Y imag()
#define CRS(a, b) (conj(a) * (b)).Y //scalar cross product
#define DOT(a, b) (conj(a) * (b)).X //dot product
#define U(p) ((p) / abs(p)) //unit vector in direction of p (
//don't use if Z(p) == true)
#define Z(x) (abs(x) < EPS)
#define A(a) (a).begin(), (a).end() //shortens sort(),
//upper_bound(), etc. for vectors

//constants (INF and EPS may need to be modified)
ld PI = acosl(-1), INF = 1e20, EPS = 1e-12;
pt I = {0, 1};

//true if d1 and d2 parallel (zero vectors considered parallel
//to everything)
bool parallel(pt d1, pt d2) { return Z(d1) || Z(d2) || Z(CRS(U(
d1), U(d2))); }

//"above" here means if l & p are rotated such that l.D points
//in the +x direction, then p is above l. Returns arbitrary
//boolean if p is on l
bool above_line(pt p, line l) { return CRS(p - l.P, l.D) > 0; }

//true if p is on line l
bool on_line(pt p, line l) { return parallel(l.P - p, l.D) &&
(!l.S || DOT(l.P - p, l.P + l.D - p) <= EPS); }

//returns 0 for no intersection, 2 for infinite intersections,
//1 otherwise. p holds intersection pt
ll intsct(line l1, line l2, pt& p) {
    if(parallel(l1.D, l2.D)) //note that two parallel segments
//sharing one endpoint are considered to have infinite
//intersections here
        return 2 * (on_line(l1.P, l2) || on_line(l1.P + l1.D, l2)
|| on_line(l2.P, l1) || on_line(l2.P + l2.D, l1));
    pt q = l1.P + l1.D * CRS(l2.D, l2.P - l1.P) / CRS(l2.D, l1.D)
;
    if(on_line(q, l1) && on_line(q, l2)) { p = q; return 1; }
    return 0;
}
```

```
//closest pt on l to p
pt cl_pt_on_l(pt p, line l) {
    pt q = l.P + DOT(U(l.D), p - l.P) * U(l.D);
    if(on_line(q, l)) return q;
    return abs(p - l.P) < abs(p - l.P - l.D) ? l.P : l.P + l.D;
}

//distance from p to l
ld dist_to(pt p, line l) { return abs(p - cl_pt_on_l(p, l)); }

//p reflected over l
pt refl_pt(pt p, line l) { return conj((p - l.P) / U(l.D)) * U(l.D) + l.P; }

//ray r reflected off l (if no intersection, returns original ray)
line reflect_line(line r, line l) {
    pt p; if(intsct(r, l, p) - 1) return r;
    return line(p, p + INF * (p - refl_pt(r.P, l)), 1);
}

//altitude from p to l
line alt(pt p, line l) { l.S = 0; return line(p, cl_pt_on_l(p, l)); }

//angle bisector of angle abc
line ang_bis(pt a, pt b, pt c) { return line(b, b + INF * (U(a - b) + U(c - b)), 1); }

//perpendicular bisector of l (assumes l.S == 1)
line perp_bis(line l) { return line(l.P + l.D / (ld)2, arg(l.D * I)); }

//orthocenter of triangle abc
pt orthocent(pt a, pt b, pt c) { pt p; intsct(alt(a, line(b, c)), alt(b, line(a, c)), p); return p; }

//incircle of triangle abc
circ incirc(pt a, pt b, pt c) {
    pt cent; intsct(ang_bis(a, b, c), ang_bis(b, a, c), cent);
    return {cent, dist_to(cent, line(a, b))};
}

//circumcircle of triangle abc
circ circumcirc(pt a, pt b, pt c) {
    pt cent; intsct(perp_bis(line(a, b, 1)), perp_bis(line(a, c, 1)), cent);
    return {cent, abs(cent - a)};
}

//is pt p inside the (not necessarily convex) polygon given by poly
bool in_poly(pt p, vector<pt>& poly) {
    line l = line(p, {INF, INF * PI}, 1);
    bool ans = false;
    pt lst = poly.back(), tmp;
    for(pt q : poly) {
        line s = line(q, lst, 1); lst = q;
        if(on_line(p, s)) return false; //change if border included
        else if(intsct(l, s, tmp)) ans = !ans;
    }
    return ans;
}

//area of polygon, vertices in order (cw or ccw)
ld area(vector<pt>& poly) {
    ld ans = 0;
    pt lst = poly.back();
    for(pt p : poly) ans += CRS(lst, p), lst = p;
}
```

```
return abs(ans / 2);
}

//perimeter of polygon, vertices in order (cw or ccw)
ld perim(vector<pt>& poly) {
    ld ans = 0;
    pt lst = poly.back();
    for(pt p : poly) ans += abs(lst - p), lst = p;
    return ans;
}

//centroid of polygon, vertices in order (cw or ccw)
pt centroid(vector<pt>& poly) {
    ld area = 0;
    pt lst = poly.back(), ans = {0, 0};
    for(pt p : poly) {
        area += CRS(lst, p);
        ans += CRS(lst, p) * (lst + p) / (ld)3;
        lst = p;
    }
    return ans / area;
}

//invert a point over a circle (doesn't work for center of circle)
pt circInv(pt p, circ c) {
    return c.R * c.R / conj(p - c.C) + c.C;
}

//vector of intersection pts of two circs (up to 2) (if circles same, returns empty vector)
vector<pt> intsctCC(circ c1, circ c2) {
    if(c1.R < c2.R) swap(c1, c2);
    pt d = c2.C - c1.C;
    if(Z(abs(d) - c1.R - c2.R)) return {c1.C + polar(c1.R, arg(c2.C - c1.C))};
    if(!Z(d) && Z(abs(d) - c1.R + c2.R)) return {c1.C + c1.R * U(d)};
    if(abs(abs(d) - c1.R) >= c2.R - EPS) return {};
    ld th = acos1((c1.R * c1.R + norm(d) - c2.R * c2.R) / (2 * c1.R * abs(d)));
    return {c1.C + polar(c1.R, arg(d) + th), c1.C + polar(c1.R, arg(d) - th)};
}

//vector of intersection pts of a line and a circ (up to 2)
vector<pt> intsctCL(circ c, line l) {
    vector<pt> v, ans;
    if(parallel(l.D, c.C - l.P)) v = {c.C + c.R * U(l.D), c.C - c.R * U(l.D)};
    else v = intsctCC(c, circ{refl_pt(c.C, l), c.R});
    for(pt p : v) if(on_line(p, l)) ans.push_back(p);
    return ans;
}

//external tangents of two circles (negate c2.R for internal tangents)
vector<line> circTangents(circ c1, circ c2) {
    pt d = c2.C - c1.C;
    ld dr = c1.R - c2.R, d2 = norm(d), h2 = d2 - dr * dr;
    if(Z(d2) || h2 < 0) return {};
    vector<line> ans;
    for(ld sg : {-1, 1}) {
        pt u = (d * dr + d * I * sqrt(h2)) * sg / d2;
        ans.push_back(line(c1.C + u * c1.R, c2.C + u * c2.R, 1));
    }
    if(Z(h2)) ans.pop_back();
    return ans;
}
```

closest-pair-of-points.h

```
Description: TODO
85eeeb, 67 lines

struct pt {
    int x, y, id;
};

struct cmp_x {
    bool operator()(const pt & a, const pt & b) const {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    }
};

struct cmp_y {
    bool operator()(const pt & a, const pt & b) const {
        return a.y < b.y;
    }
};

int n;
vector<pt> a;

double mindist;
pair<int, int> best_pair;

void upd_ans(const pt & a, const pt & b) {
    double dist = sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y));
    if (dist < mindist) {
        mindist = dist;
        best_pair = {a.id, b.id};
    }
}

vector<pt> t;

void rec(int l, int r) {
    if (r - l <= 3) {
        for (int i = l; i < r; ++i) {
            for (int j = i + 1; j < r; ++j) {
                upd_ans(a[i], a[j]);
            }
        }
        sort(a.begin() + l, a.begin() + r, cmp_y());
        return;
    }

    int m = (l + r) >> 1;
    int midx = a[m].x;
    rec(l, m);
    rec(m, r);

    merge(a.begin() + l, a.begin() + m, a.begin() + m, a.begin() + r, t.begin(), cmp_y());
    copy(t.begin(), t.begin() + r - l, a.begin() + l);

    int tsz = 0;
    for (int i = l; i < r; ++i) {
        if (abs(a[i].x - midx) < mindist) {
            for (int j = tsz - 1; j >= 0 && a[i].y - t[j].y < mindist; --j)
                upd_ans(a[i], t[j]);
            t[tsz++] = a[i];
        }
    }
}

void call_rec(){
    t.resize(n);
}
```

```
sort(a.begin(), a.end(), cmp_x());
mindist = 1E20;
rec(0, n);
}
```

2.10 Advanced Algorithms

fft.h

Description: TODO

cbf4b6, 63 lines

```
//from cp algorithm the inplace-computation fft part
using cd = complex<double>;
const double PI = acos(-1);

int reverse(int num, int lg_n) {
    int res = 0;
    for (int i = 0; i < lg_n; i++) {
        if (num & (1 << i))
            res |= 1 << (lg_n - 1 - i);
    }
    return res;
}

void fft(vector<cd> & a, bool invert) {
    int n = a.size();
    int lg_n = 0;
    while ((1 << lg_n) < n)
        lg_n++;

    for (int i = 0; i < n; i++) {
        if (i < reverse(i, lg_n))
            swap(a[i], a[reverse(i, lg_n)]);
    }

    for (int len = 2; len <= n; len <= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }

    if (invert) {
        for (cd & x : a)
            x /= n;
    }
}

vector<int> multiply(vector<int> const& a, vector<int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)a.size() + (int)b.size())
        n <= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);

    vector<int> result(n);
```

```
for (int i = 0; i < n; i++)
    result[i] = round(fa[i].real());
return result;
}

mo2D.h
Description: TODO
9df2fb, 49 lines

/* always use zero based indexing*/
/* there N/B blocks so left move ~ N/B*B + Q*B, right move ~ N/
    B*B + (N/B)*B */
/* so O(N^2/B + N + QB) ~ O((N+Q)sqrt(N)) when B = sqrt(N) */
/* when Q~N it's ~ O(N^3/2) */
void remove(int idx); // TODO: remove value at idx from data
    structure
void add(int idx); // TODO: add value at idx from data
    structure
int get_answer(); // TODO: extract the current answer of the
    data structure

int block_size;

struct Query {
    int l, r, idx;
    bool operator<(Query other) const
    {
        return make_pair(l / block_size, r) <
            make_pair(other.l / block_size, other.r);
    }
};

vector<int> mo_s_algorithm(vector<Query>& queries) {
    vector<int> answers(queries.size());
    sort(queries.begin(), queries.end());

    // TODO: initialize data structure

    int cur_l = 0;
    int cur_r = -1;
    // invariant: data structure will always reflect the range
        [cur_l, cur_r]
    for (Query q : queries) {
        while (cur_l > q.l) {
            cur_l--;
            add(cur_l);
        }
        while (cur_r < q.r) {
            cur_r++;
            add(cur_r);
        }
        while (cur_l < q.l) {
            remove(cur_l);
            cur_l++;
        }
        while (cur_r > q.r) {
            remove(cur_r);
            cur_r--;
        }
        answers[q.idx] = get_answer();
    }
    return answers;
}

mo3D.h
Description: TODO
dc4a5e, 62 lines

/* always use zero based indexing*/
/* ~ O(N^(5/3)) */
int block_size;
```

```
struct Query {
    int l, r, t, idx;
    bool operator<(Query other) const
    {
        return make_tuple(l / block_size, r / block_size, t) <
            make_tuple(other.l / block_size, other.r /
                block_size, other.t);
    }
};

int ans[N];
void mo_s_algorithm(vector<Query>& queries) {
    sort(queries.begin(), queries.end());
    // TODO: initialize data structure

    int cur_l = 0;
    int cur_r = -1;
    int cur_t = -1;
    // invariant: data structure will always reflect the range
        [cur_l, cur_r]
    for (Query q : queries) {
        while (cur_l > q.l) {
            cur_l--;
            //add(cur_l)
        }
        while (cur_r < q.r) {
            cur_r++;
            //add(cur_r)
        }
        while (cur_l < q.l) {
            //remove(cur_l);
            cur_l++;
        }
        while (cur_r > q.r) {
            //remove(cur_r);
            cur_r--;
        }

        while(cur_t < q.t){
            cur_t++;
            //update
        }
        while(cur_t > q.t){
            //update rollback
            cur_t--;
        }
        ans[q.idx] = get_answer(); // declare array for this
    }
}

void getBlockSize(int n, int q){
    if (q >= n) {
        // most common: Q ~ N
        block_size = max(1, (int)pow(n, 2.0/3.0));
    } else {
        // general cube-root rule
        block_size = max(1, (int)pow((long double)n*n / q,
            1.0/3.0));
    }
    // then sort your queries by (L/B, R/B, T) or via Hilbert
}

divide-and-conquer-dp.h
Description: TODO
108b88, 55 lines

//TODO: initialize dp , initialize cost() function of use slide
    ()
//ll dp[N][M];
```

```
//generic implementation for sliding range technique for logn
cost()
//(persistent segtree alternative)
ll ccost = 0;
int cl = 0, cr = -1;
void slide(int l, int r){
    while(cr < r){
        ++cr;
        //add();
        //...
    }
    while(cl > l){
        --cl;
        //add();
        //...
    }
    while(cr > r){
        //remove();
        //...
        --cr;
    }
    while(cl < l){
        //remove();
        //...
        ++cl;
    }
}

void compute(int l, int r, int optl, int opt, int j){
    if(l>r) return;

    int mid = (l+r)>>1;
    //pair<ll, int> best = {0,-1};
    //pair<ll, int> best = {LINF,-1};

    //dp is satisfy quadrangle IE if cost() satisfy qudrangle
    IE
    //if cost() is QF => opt() is nondecreasing
    for(int k=optl;k<=min(mid,opt);++k){
        slide(k,mid);
        //best = max(best, {(k>0)?dp[k-1][j-1]:0} + ccost, k );
        //best = min(best, {(k>0)?dp[k-1][j-1]:0} + ccost, k );
    }

    //dp[mid][j] = max(dp[mid][j], best.first);
    //dp[mid][j] = min(dp[mid][j], best.first);
    int opt = best.second;
    if(l!=r){
        compute(l,mid-1,optl,opt,j);
        compute(mid+1,r,opt,opt,j);
    }
}

//TODO: set dp to LINF or -LINF
```

cellul4r (3)

2SAT.h

Description: solve 2 sat form of or to implies

Time: $O(N + M)$

025007, 70 lines

```
struct TwoSatSolver {
    int n_vars;
    int n_vertices;
    vector<vector<int>> adj, adj_t;
```

```
vector<bool> used;
vector<int> order, comp;
vector<bool> assignment;
TwoSatSolver(int n_vars) : n_vars(n_vars), n_vertices(2 *
    n_vars), adj(n_vertices), adj_t(n_vertices), used(
    n_vertices), order(), comp(n_vertices, -1), assignment
    (n_vars) {
    order.reserve(n_vertices);
}
void dfs1(int v) {
    used[v] = true;
    for (int u : adj[v]) {
        if (!used[u])
            dfs1(u);
    }
    order.push_back(v);
}
void dfs2(int v, int cl) {
    comp[v] = cl;
    for (int u : adj_t[v]) {
        if (comp[u] == -1)
            dfs2(u, cl);
    }
}
bool solve_2SAT() {
    order.clear();
    used.assign(n_vertices, false);
    for (int i = 0; i < n_vertices; ++i) {
        if (!used[i])
            dfs1(i);
    }
    comp.assign(n_vertices, -1);
    for (int i = 0, j = 0; i < n_vertices; ++i) {
        int v = order[n_vertices - i - 1];
        if (comp[v] == -1)
            dfs2(v, j++);
    }

    assignment.assign(n_vars, false);
    for (int i = 0; i < n_vertices; i += 2) {
        if (comp[i] == comp[i + 1])
            return false;
        assignment[i / 2] = comp[i] > comp[i + 1];
    }
    return true;
}

void add_disjunction(int a, bool na, int b, bool nb) {
    // na and nb signify whether a and b are to be negated
    a = 2 * a ^ na;
    b = 2 * b ^ nb;
    int neg_a = a ^ 1;
    int neg_b = b ^ 1;
    adj[neg_a].push_back(b);
    adj[neg_b].push_back(a);
    adj_t[b].push_back(neg_a);
    adj_t[a].push_back(neg_b);
}

static void example_usage() {
    TwoSatSolver solver(3);
    solver.add_disjunction(0, false, 1, true);
    //      a or not b
    solver.add_disjunction(0, true, 1, true);    // not a
    //      or not b
    solver.add_disjunction(1, false, 2, false); //      b
    //      or c
    solver.add_disjunction(0, false, 0, false); //      a
    //      or a
    assert(solver.solve_2SAT() == true);
    auto expected = vector<bool>(True, False, True);
```

```
    assert(solver.assignment == expected);
}
};

kadane2D.h
Time:  $O()$ 
6227c1, 59 lines

int kadane(vector<int>& temp) {
    int rows = temp.size();
    int currSum = 0;
    int maxSum = INT_MIN;

    for (int i = 0; i < rows; i++) {
        currSum += temp[i];

        // Update maxSum if the current sum is greater
        if (maxSum < currSum) {
            maxSum = currSum;
        }

        // If the current sum becomes negative, reset it to 0
        if (currSum < 0) {
            currSum = 0;
        }
    }

    return maxSum;
}

// Function to find the maximum sum rectangle in a 2D matrix
int maxRectSum(vector<vector<int>> &mat) {
    int rows = mat.size();
    int cols = mat[0].size();

    int maxSum = INT_MIN;

    // Initialize a temporary array to store row-wise
    // sums between left and right boundaries
    vector<int> temp(rows);

    // Check for all possible left and right boundaries
    for (int left = 0; left < cols; left++) {

        // Reset the temporary array for each new left boundary
        for (int i = 0; i < rows; i++)
            temp[i] = 0;

        for (int right = left; right < cols; right++) {

            // Update the temporary array with the current
            // column's values
            for (int row = 0; row < rows; row++) {
                temp[row] += mat[row][right];
            }

            // Find the maximum sum of the subarray for the
            // current column boundaries
            int sum = kadane(temp);

            // Update the maximum sum found so far
            maxSum = max(maxSum, sum);
        }
    }

    return maxSum;
}
```



```
int lcs(string &s1, string &s2) {
    int m = s1.size();
    int n = s2.size();

    // Initializing a matrix of size (m+1)*(n+1)
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    // Building dp[m+1][n+1] in bottom-up fashion
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            if (s1[i - 1] == s2[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    // dp[m][n] contains length of LCS for s1[0..m-1]
    // and s2[0..n-1]
    return dp[m][n];
}

int lcs(string &s1, string &s2) {

    int m = s1.length(), n = s2.length();

    // dp vector is initialized to all zeros
    // This vector stores the LCS values for the current row.
    // dp[j] represents LCS of s1[0..i] and s2[0..j]
    vector<int> dp(n + 1, 0);

    // i and j represent the lengths of s1 and s2 respectively
    for (int i = 1; i <= m; ++i) {

        // prev stores the value from the previous
        // row and previous column (i-1), (j -1)
        // Used to keep track of LCS[i-1][j-1] while updating
        // dp[j]
        int prev = dp[0];

        for (int j = 1; j <= n; ++j) {

            // temp temporarily stores the current
            // dp[j] before it gets updated
            int temp = dp[j];

            // If characters match, add 1 to the value
            // from the previous row and previous column
            // dp[j] = 1 + LCS[i-1][j-1]
            if (s1[i - 1] == s2[j - 1])
                dp[j] = 1 + prev;
            else
                // Otherwise, take the maximum of the
                // left (dp[j-1]) and top (dp[j]) values
                dp[j] = max(dp[j - 1], dp[j]);

            // Update prev for the next iteration
            // This keeps the value of the previous
            // row (i-1) for future comparisons
            prev = temp;
        }
    }

    // The last element of the vector contains the length of
    // the LCS
    // dp[n] stores the length of LCS of s1[0..m] and s2[0..n]
```

```
        return dp[n];
    }

    // Function that returns the number of paths
    int numberOfPaths(int source, int destination, int n, int fre
        [])
    {

        // make topological sorting
        vector<int> s = topological_sorting(fre, n);

        // to store required answer.
        int dp[n] = { 0 };

        // answer from destination
        // to destination is 1.
        dp[destination] = 1;

        // traverse in reverse order
        for (int i = s.size() - 1; i >= 0; i--) {
            for (int j = 0; j < v[s[i]].size(); j++) {
                dp[s[i]] += dp[v[s[i]][j]];
            }
        }

        return dp[source];
    }
```

Mathematics (4)

4.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e & x &= \frac{ed - bf}{ad - bc} \\ cx + dy &= f & y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1n + d_2)r^n$.

4.2 Trigonometry

$$\begin{aligned} \sin(v + w) &= \sin v \cos w + \cos v \sin w \\ \cos(v + w) &= \cos v \cos w - \sin v \sin w \end{aligned}$$

$$\begin{aligned} \tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \end{aligned}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi) \end{aligned}$$

where $r = \sqrt{a^2 + b^2}, \phi = \operatorname{atan2}(b, a)$.

4.3 Geometry

4.3.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles):

$m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b + c} \right)^2 \right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

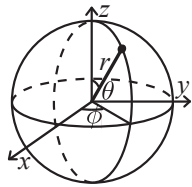
4.3.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° , $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

4.3.3 Spherical coordinates



$$\begin{aligned}x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\z &= r \cos \theta & \phi &= \operatorname{atan2}(y, x)\end{aligned}$$

4.4 Derivatives/Integrals

$$\begin{aligned}\frac{d}{dx} \arcsin x &= \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx} \arccos x &= -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx} \tan x &= 1 + \tan^2 x & \frac{d}{dx} \arctan x &= \frac{1}{1+x^2} \\ \int \tan ax &= -\frac{\ln |\cos ax|}{a} & \int x \sin ax &= \frac{\sin ax - ax \cos ax}{a^2} \\ \int e^{-x^2} &= \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) & \int x e^{ax} dx &= \frac{e^{ax}}{a^2} (ax - 1)\end{aligned}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

4.5 Sums

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$\begin{aligned}1 + 2 + 3 + \dots + n &= \frac{n(n+1)}{2} \\ 1^2 + 2^2 + 3^2 + \dots + n^2 &= \frac{n(2n+1)(n+1)}{6} \\ 1^3 + 2^3 + 3^3 + \dots + n^3 &= \frac{n^2(n+1)^2}{4} \\ 1^4 + 2^4 + 3^4 + \dots + n^4 &= \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}\end{aligned}$$

4.6 Series

$$\begin{aligned}e^x &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty) \\ \ln(1+x) &= x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1) \\ \sqrt{1+x} &= 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1)\end{aligned}$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)$$

4.7 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2 V(X) + b^2 V(Y).$$

4.7.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\operatorname{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np, \sigma^2 = np(1-p)$$

$\operatorname{Bin}(n, p)$ is approximately $\operatorname{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\operatorname{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1}, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1-p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\operatorname{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$

$$\mu = \lambda, \sigma^2 = \lambda$$

4.7.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\operatorname{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2}, \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\operatorname{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

Number Theoretic Functions

Md Nafis Ul Haque Shifat

October 23, 2023

Contents

1	Multiplicative Function	2
2	Dirichlet Convolution	2
3	More on common multiplicative functions	5
3.1	Number-of-Divisors Function	5
3.2	Sum-of-Divisors Function	5
3.3	Möbius Function	5
3.4	Euler's Totient Function	6
4	The Möbius Inversion	7
5	Problems	7

1

If $mn > 1$ then

$$\begin{aligned}
 h(mn) &= \sum_{d|mn} f(d)g(mn/d) \\
 &= \sum_{a|n, b|n} f(ab)g(mn/ab) \\
 &= \sum_{a|n, b|n, ab < mn} f(a)f(b)g(m/a)g(n/b) + f(mn)g(1) \\
 &= \sum_{a|n, b|n} f(a)f(b)g(m/a)g(n/b) - f(m)f(n) + f(mn) \\
 &= \sum_{a|n} f(a)g(m/a) \sum_{b|n} f(b)g(n/b) - f(m)f(n) + f(mn) \\
 &= h(m)h(n) - f(m)f(n) + f(mn)
 \end{aligned}$$

Now if h is multiplicative, $f(m)f(n) = f(mn)$, which implies f is multiplicative as well. $\square$

Definition 2.2 (Dirichlet Inverse). If f is an arithmetic function, we call g **dirichlet inverse** of f , denoted by f^{-1} , if $f * g = \varepsilon$.

Recall that ε is the unit function: $\varepsilon(n) = \begin{cases} 1 & \text{if } n = 1 \\ 0 & \text{if } n > 1 \end{cases}$

Theorem 4. If f is an arithmetic function where $f(1) \neq 0$, then f^{-1} exists.

Proof. Let $g = \begin{cases} \frac{1}{f(1)} & \text{if } n = 1 \\ -\frac{\sum_{d|n, d < n} f(n/d)g(d)}{f(1)} & \text{if } n > 1 \end{cases}$

Clearly, $(f * g)(1) = 1$. Now for $n > 1$,

$$\begin{aligned}
 (f * g)(n) &= (g * f)(n) \\
 &= \sum_{d|n} g(d)f(n/d) \\
 &= g(n)f(1) + \sum_{d|n, d < n} g(d)f(n/d)
 \end{aligned}$$

Now,

$$g(n) = -\frac{\sum_{d|n, d < n} g(d)f(n/d)}{f(1)} \implies -g(n)f(1) = \sum_{d|n, d < n} g(d)f(n/d)$$

So, $(f * g)(n) = g(n)f(1) - g(n)f(1) = 0$. Hence, $(f * g) = \varepsilon$, thus $g = f^{-1}$. $\square$

Corollary 2. If f is multiplicative, so is f^{-1}

Proof. Since $\varepsilon = f * f^{-1}$, and both ε and f are multiplicative, by **theorem 3**, f^{-1} is also multiplicative. $\square$

4

1 Multiplicative Function

Definition 1.1 (Multiplicative Function). A number-theoretic function f is **multiplicative** if $f(1) = 1$ and $f(mn) = f(m)f(n)$, $\forall m, n \in \mathbb{N}$ such that $\gcd(m, n) = 1$. Additionally, f is called **completely multiplicative** if $f(mn) = f(m)f(n)$, $\forall m, n \in \mathbb{N}$.

Examples.

- $1(n) = 1$: The constant function.
- $Id(n) = n$: The identity function.

- $\varepsilon(n) = \begin{cases} 1 & \text{if } n = 1 \\ 0 & \text{if } n > 1 \end{cases}$: The unit function

- $\tau(n) = \sum_{d|n} 1$: The number of divisors function.

- $\sigma(n) = \sum_{d|n} d$: The sum of divisors function.

- $\phi(n) = \sum_{i=1}^n [\gcd(i, n) = 1]$: Euler's Totient Function (here the third brackets serve as a boolean function, which returns 1 if the condition is true, 0 otherwise.)

Note that $1, Id, \varepsilon$ are completely multiplicative as well, while ϕ, τ, σ aren't.

Theorem 1. If f is a multiplicative function and if $n = \prod_{i=1}^r p_i^{e_i}$, then $f(n) = \prod_{i=1}^r f(p_i^{e_i})$.

Proof. Since $\gcd(p_i^{e_i}, p_j^{e_j}) = 1, \forall i \neq j$, induction on r proves the theorem. $\square$

2 Dirichlet Convolution

Definition 2.1 (Dirichlet Convolution). If f and g are two arithmetic functions, then their **dirichlet convolution**, denoted by $f * g$, is defined as

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d)$$

Alternatively, we can write $(f * g)(n) = \sum_{ab=n} f(a)g(b)$.

Properties of Dirichlet Convolution

- Convolution is Commutative: $f * g = g * f$
- It is Associative: $(f * g) * h = f * (g * h)$

$$\text{Proof. } ((f * g) * h)(n) = \sum_{d|n} (f * g)(d)h(n/d) = \sum_{d|n} \sum_{ab=d} f(a)g(b)h(n/d) = \sum_{abc=n} f(a)g(b)h(c)$$

$$\text{Similarly, } (f * (g * h))(n) = \sum_{ad=n} f(a)(g * h)(d) = \sum_{ad=n} f(a) \sum_{bc=d} g(b)h(c) = \sum_{abc=n} f(a)g(b)h(c)$$

$\square$

2

3 More on common multiplicative functions

3.1 Number-of-Divisors Function

Let $\tau(n)$ denote the number of divisors of n . So, $\tau(n) = \sum_{d|n} 1$. It is easy to see that τ is multiplicative, since $\tau(n) = \sum_{d|n} 1 = \sum_{d|n} 1(d)1(n/d) = (1 * 1)(n)$. Recall $1(n)$ is the constant function, which is completely multiplicative. Hence, by **theorem 2**, τ is also multiplicative.

Now it is easy to derive the formula of this function. What is $\tau(p^x)$ where p is prime? Of course the divisors of p^x are $1, p, p^2, \dots, p^x$, so total $(x + 1)$ divisors. Now for any $n = \prod_{i=1}^r p_i^{e_i}$, we can simply apply **theorem 1** and get $\tau(n) = \prod_{i=1}^r (e_i + 1)$. $\tau(n)$ can be calculated with a simple loop in $O(\sqrt{n})$.

3.2 Sum-of-Divisors Function

The sum of divisors function is denoted by σ . We can write $\sigma(n) = \sum_{d|n} d$. Similar to the previous function, we can write $\sigma(n) = \sum_{d|n} d = \sum_{d|n} Id(d)1(n/d) = (Id * 1)(n)$. Since Id and 1 are both multiplicative, by **theorem 2**, σ is also multiplicative.

Now, note that for a prime p , $\sigma(p^x) = 1 + p + p^2 + \dots + p^x = \frac{p^{x+1} - 1}{p - 1}$. So, for $n = \prod_{i=1}^r p_i^{e_i}$, $\sigma(n) = \prod_{i=1}^r \frac{p_i^{e_i+1} - 1}{p_i - 1}$.

3.3 Möbius Function

Definition 3.1. For a positive integer n , the **möbius function**, denoted by μ , is defined as,

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1 \\ 0 & \text{if } p^2 | n \text{ for some prime } p \\ (-1)^r & \text{if } n = p_1 p_2 \dots p_r, \text{ where } p_i \text{ are distinct primes} \end{cases}$$

The next theorem shows a very important property of möbius function.

Theorem 5. $\sum_{d|n} \mu(d) = \begin{cases} 1 & \text{if } n = 1 \\ 0 & \text{if } n > 1 \end{cases} = \varepsilon(n)$

Proof. For $n = 1$, $\sum_{d|n} \mu(d) = \mu(1) = 1$
When $n > 1$, let $n = \prod_{i=1}^r p_i^{e_i}$. If $d|n$, then $d = \prod_{i=1}^r p_i^{f_i}$, where $f_i \leq e_i$, for $1 \leq i \leq r$. Now if $\exists i$ such that $f_i > 1$, $\mu(d)$ will be 0. So, the only divisors we care about are the ones with $f_i \leq 1, \forall i$. This means we have r distinct primes, and we will go through each possible combinations and add their contribution to the sum accordingly. For example, if d contains k primes, then we have $\binom{r}{k}$

5

- $f * \varepsilon = f$

Proof.

$$(f * \varepsilon)(n) = \sum_{d|n} f(d)\varepsilon(n/d)$$

Now if $d < n$, $n/d > 1$, so $\varepsilon(n/d) = 0$. Therefore

$$(f * \varepsilon)(n) = f(n)\varepsilon(1) = f(n)$$

$\square$

Theorem 2. if f and g are both multiplicative, so is $f * g$.

Proof. Let $h = f * g$. Now, if $\gcd(m, n) = 1$,

$$h(mn) = \sum_{d|mn} f(d)g(mn/d)$$

Now since $d|mn$ and $\gcd(m, n) = 1$, $d = ab$ where $a|m$, $b|n$ and $\gcd(a, b) = 1$. Hence,

$$\begin{aligned}
 h(mn) &= \sum_{a|m, b|n} f(ab)g(mn/ab) \\
 &= \sum_{a|m, b|n} f(a)f(b)g(m/a)g(n/b) \\
 &= \sum_{a|m} f(a)g(m/a) \sum_{b|n} f(b)g(n/b) \\
 &= h(m)h(n)
 \end{aligned}$$

$\square$

Corollary 1. if f is a multiplicative function, and $F(n) = \sum_{d|n} f(n)$, F is multiplicative as well.

Proof. $F(n) = \sum_{d|n} f(n) = \sum_{d|n} f(n)1(\frac{n}{d}) = (f * 1)(n)$.

Since $f, 1$ both are multiplicative, by **theorem 2**, F is also multiplicative. $\square$

Theorem 3. if $h = f * g$ and h, g are both multiplicative, so is f .

Proof. Suppose f is not multiplicative. So, there exists a pair of positive integers (m, n) with $\gcd(m, n) = 1$ such that $f(mn) \neq f(m)f(n)$. We take such a pair with smallest mn .

If $mn = 1$ then $f(1) \neq f(1)f(1)$ which implies $f(1) \neq 1$. Now since g is multiplicative, $g(1) = 1$. But $h(1) = (f * g)(1) = f(1)g(1) \neq 1$ since $f(1) \neq 1$. This is a contradiction since h is multiplicative and $h(1) = 1$.

3

possible values for d , and each of them will contribute $(-1)^k$ to the sum. So,

$$\begin{aligned}
 \sum_{d|n} \mu(d) &= \sum_{\substack{(f_1, f_2, \dots, f_r) \\ f_i \in \{0, 1\}}} \mu\left(\prod_{i=1}^r p_i^{f_i}\right) \\
 &= (-1)^0 \binom{r}{0} + (-1)^1 \binom{r}{1} + \dots + (-1)^r \binom{r}{r} \\
 &= \sum_{i=0}^r \binom{r}{i} (-1)^i \\
 &= (1 - 1)^r \\
 &= 0
 \end{aligned}$$

$\square$

We also conclude that μ is **multiplicative** from this theorem, since,

$$\sum_{d|n} \mu(d) = \sum_{d|n} \mu(d)1(n/d) = (\mu * 1)(n) = \varepsilon(n)$$

This implies μ is the dirichlet inverse of the constant function. So, by **Corollary 2**, μ is multiplicative. Since μ is the dirichlet inverse of the constant function, using the definition of g in **theorem 4**, we can write that,

$$\mu(n) = \sum_{d|n, d < n} -\mu(d)$$

Now we can use this in sieve to calculate the value of möbius function for integers from 1 to n . Here's an implementation in C++:

```

mu[1] = 1;
for(int i = 1; i < N; i++) {
    for(int j = 2 * i; j < N; j += i) {
        mu[j] -= mu[i];
    }
}

```

This is basically the standard sieve: instead of iterating over d for each n , for each d , we loop over its multiples and add its contribution to the answer of the multiples. So, it runs in $O(n \log n)$ time.

3.4 Euler's Totient Function

Definition 3.2. For a positive integer n , **Euler's Totient Function**, denoted as $\phi(n)$, is defined as the number of positive integers i up to n such that $\gcd(i, n) = 1$.

For example, $\phi(4) = 2$, since from 1 to 4, only 1 and 3 are coprimes with 4.

Theorem 6. $\sum_{d|n} \phi(d) = n$

Proof. Let S_d be the set of all positive integers up to n whose gcd with n is d . Now, $\gcd(n, m) = d$ implies $\gcd(n/d, m/d) = 1$. By definition, the number of such m is $\phi(n/d)$. So,

$$\sum_{d|n} |S_d| = \sum_{d|n} \phi(n/d)$$

6

Clearly, the sets S_d for all d are pairwise disjoint. Moreover, for every integer $1 \leq k \leq n$, there exists a d such that $d|n$ and $k \in S_d$. Hence, $\sum_{d|n} |S_d| = \sum_{d|n} \phi(n/d) = \sum_{d|n} \phi(d) = n$. $\square$

Since, $\sum_{d|n} \phi(d) = \sum_{d|n} \phi(d)1(n/d) = (\phi * 1)(n) = Id(n)$, we also see that ϕ is a multiplicative function.

Note that if p is prime, $\phi(p) = p - 1$, and $\phi(p^k) = p^k - p^{k-1} = p^{k-1}(p - 1)$. So, if $n = \prod_{i=1}^r p_i^{e_i}$, by **theorem 1**, we get

$$\phi(n) = \prod_{i=1}^r p_i^{e_i-1}(p_i - 1) = \prod_{i=1}^r p_i^{e_i} \frac{(p_i - 1)}{p_i} = n \prod_{i=1}^r \left(1 - \frac{1}{p_i}\right)$$

which is the well-known formula for calculating $\phi(n)$.

About implementation, we can simply initialize $\phi(n) = n$, and for each prime divisor p , we update its value by multiplying it with $(1 - \frac{1}{p}) = \frac{p-1}{p}$. Here's the code:

```
for (int i = 1; i <= n; i++) phi[i] = i;
for (int p = 2; p <= n; p++) {
    if (phi[p] == p) {
        phi[p] = p - 1;
        for (int i = 2 * p; i <= n; i += p) {
            phi[i] = (phi[i] / p) * (p - 1);
        }
    }
}
```

4 The Möbius Inversion

Theorem 7. Let f and g be two arithmetic functions. Then, $g(n) = \sum_{d|n} f(d)$ if and only if $f(n) = \sum_{d|n} g(d)\mu(n/d)$.

Proof. Note that $g(n) = \sum_{d|n} f(d) = \sum_{d|n} f(d)1(n/d) = (f * 1)(n)$, which implies $g = f * 1$. Similarly, $f(n) = \sum_{d|n} g(d)\mu(n/d) = (g * \mu)(n)$, which implies $f = g * \mu$. Hence it suffices to show that $g = f * 1$ if and only if $f = g * \mu$.

$$g * \mu = (f * 1) * \mu = f * (1 * \mu) = f * \varepsilon = f$$

. Now for converse,

$$f * 1 = (g * \mu) * 1 = g * (1 * \mu) = g * \varepsilon = g$$

. I kept this single theorem in a separate section only because it is probably the most useful part of the entire note!

5 Problems

Problem 1. Given n , calculate the number of pairs (i, j) such that $i, j \in [1, n]$ and $\gcd(i, j) = 1$.

Solution. We basically need to find the value of $\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) = 1$. Now note that $|\gcd(i, j) = 1| = \varepsilon(\gcd(i, j))$. Hence,

7

Let $\text{cnt}[x]$ be the number of such tuples whose gcd is x . Also, let $n = \min_{i=1}^k n_i$. Then our answer will be $\sum_{g=1}^n g \times \text{cnt}[g]$. We can write $g = \sum_{d|g} \phi(d)$. So,

$$\sum_{g=1}^n g \times \text{cnt}[g] = \sum_{g=1}^n \sum_{d|g} \phi(d) \times \text{cnt}[g]$$

Instead of iterating over divisors d of each g , we can iterate over multiples g of each d . So,

$$\begin{aligned} \sum_{g=1}^n g \times \text{cnt}[g] &= \sum_{g=1}^n \sum_{d|g} \phi(d) \times \text{cnt}[g] \\ &= \sum_{d=1}^n \phi(d) \times \sum_{d|g} \text{cnt}[g] \end{aligned}$$

Now, $\sum_{d|g} \text{cnt}[g]$ is basically the number of tuples whose gcd is divisible by d . It is easy to see that the number of such tuples are $\lfloor \frac{n_1}{d} \rfloor \times \lfloor \frac{n_2}{d} \rfloor \times \dots \times \lfloor \frac{n_k}{d} \rfloor$. Hence the answer is,

$$\sum_{d=1}^n \phi(d) \left(\left\lfloor \frac{n_1}{d} \right\rfloor \times \left\lfloor \frac{n_2}{d} \right\rfloor \times \dots \times \left\lfloor \frac{n_k}{d} \right\rfloor \right)$$

which is calculatable in $O(nk)$.

Problem 6. (Devu and Birthday Celebration) Answer Q ($Q \leq 10^6$) queries: Given n and f ($n, f \leq 10^5$), calculate the number of ways to distribute n sweets among f friends so that if i -th friend got a_i sweets, $a_i \geq 1$ for all i and $\gcd(a_1, a_2, \dots, a_f) = 1$. Note that the ordering matters: $a = [1, 2]$ and $a = [2, 1]$ are considered different.

Solution. First of all, how many ways are there to distribute n sweets among f people so that everyone gets at least one sweet (ignoring the gcd condition)? For that, we can represent a distribution with a string of 'o' and '|', where 'o' means we give a sweet to the current person and '|' means we move to the next person. For example, if $n = 6$ and $f = 3$, then 'oo|ooo|o' denotes $a = \{2, 3, 1\}$. Since the string has n o's, we have $(n - 1)$ places to insert $(f - 1)$ |'s, which can be done in $\binom{n-1}{f-1}$ ways.

Now for a fixed f , let $h(n)$ be the number of ways to distribute n sweets among f people so that everyone gets at least one sweet. From the explanation above, we get $h(n) = \binom{n-1}{f-1}$. Now let $f(n)$ be the number of ways to distribute n sweets among f people so that each gets at least 1 and gcd of all a_i 's is 1 (which is basically the answer of the problem).

For a particular distribution, if $\gcd(a_1, a_2, \dots, a_f) = d$, then $d|n$.

$$\begin{aligned} a_1 + a_2 + \dots + a_f &= n \\ \Rightarrow d(a'_1 + a'_2 + \dots + a'_f) &= n \\ \Rightarrow a'_1 + a'_2 + \dots + a'_f &= \frac{n}{d} \end{aligned}$$

where $\gcd(a'_1, a'_2, \dots, a'_f) = 1$. Since $a'_1 + a'_2 + \dots + a'_f = \frac{n}{d}$ and $\gcd(a'_1, a'_2, \dots, a'_f) = 1$, by definition, the number of such a' is $f(n/d)$. Hence, we get

$$h(n) = \sum_{d|n} f(n/d) = \sum_{d|n} f(d)$$

$$\sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = 1] = \sum_{i=1}^n \sum_{j=1}^n \varepsilon(\gcd(i, j)) = \sum_{i=1}^n \sum_{j=1}^n \sum_{d|\gcd(i, j)} \mu(d)$$

Now, $d|\gcd(i, j)$ means $d|i$ and $d|j$. So,

$$\begin{aligned} \sum_{i=1}^n \sum_{j=1}^n \sum_{d|\gcd(i, j)} \mu(d) &= \sum_{i=1}^n \sum_{j=1}^n \sum_{d=1}^n [d \mid i][d \mid j] \mu(d) \\ &= \sum_{d=1}^n \mu(d) \sum_{i=1}^n [d \mid i] \sum_{j=1}^n [d \mid j] \\ &= \sum_{d=1}^n \mu(d) \left(\left\lfloor \frac{n}{d} \right\rfloor \right)^2 \end{aligned}$$

Note that the number of multiples of d from 1 to n is $\lfloor \frac{n}{d} \rfloor$. We can calculate the value of $\mu(i)$ for integers up to n with sieve, and calculate the final answer with a simple loop.

The sequence $\lfloor \frac{n}{1} \rfloor, \lfloor \frac{n}{2} \rfloor, \dots, \lfloor \frac{n}{n} \rfloor$ contains at most $2\sqrt{n}$ unique values, so if we precalculate the $\mu(i)$, we can calculate the answer for each n in $O(\sqrt{n})$ time.

```
for (int l = 1, r; l <= n; l = r + 1) {
    r = n / (n / l);
    //now for all l <= i <= r, n / i is equal.
}
```

Problem 2. Given n , calculate the number of pairs (i, j) such that $1 \leq i < j \leq n$ and $\gcd(i, j) = 1$.

Solution. We need to calculate $\sum_{i=1}^n \sum_{j=i+1}^n |\gcd(i, j) = 1|$.

$$\begin{aligned} \sum_{i=1}^n \sum_{j=i+1}^n [\gcd(i, j) = 1] &= \sum_{j=2}^n \sum_{i=1}^{j-1} [\gcd(i, j) = 1] \\ &= \sum_{j=2}^n \phi(j) \end{aligned}$$

Now it's a simple loop!

Problem 3. Given a, b, k , calculate the number of pairs (x, y) such that $x \in [1, a]$, $y \in [1, b]$ and $\gcd(x, y) = k$. Also, (x, y) and (y, x) are considered the same.

Solution. First of all, note that $\gcd(x, y) = k$ implies that $\gcd(x/k, y/k) = 1$. So, we can rephrase the statement: calculate the number of pairs (x, y) such that $x \in [1, \lfloor \frac{a}{k} \rfloor]$, $y \in [1, \lfloor \frac{b}{k} \rfloor]$ and $\gcd(x, y) = 1$.

Let $n = \lfloor \frac{a}{k} \rfloor$ and $m = \lfloor \frac{b}{k} \rfloor$. For simplicity, let's assume $n \geq m$ (otherwise we can simply swap n, m). We need to calculate $\sum_{i=1}^n \sum_{j=1}^m |\gcd(i, j) = 1|$.

8

Finally, we can apply Möbius inversion!

$$f(n) = \sum_{d|n} h(d)\mu(n/d) = \sum_{d|n} \left(\frac{d-1}{d} \right) \mu(n/d)$$

We can simply precalculate the $\mu(i)$ and the divisors for integer up to 10^5 to answer the queries efficiently.

Problem 7. (COPRIME3) Given an array a of length n ($1 \leq a[i] \leq 10^6$, $1 \leq n \leq 10^5$), calculate the number of triplets $i < j < k$ such that $\gcd(a[i], a[j], a[k]) = 1$.

Solution. We basically need to calculate $\sum_{i=1}^n \sum_{j=i+1}^n \sum_{k=j+1}^n |\gcd(a[i], a[j], a[k]) = 1|$. So,

$$\begin{aligned} \sum_{i=1}^n \sum_{j=i+1}^n \sum_{k=j+1}^n [\gcd(a[i], a[j], a[k]) = 1] &= \sum_{i=1}^n \sum_{j=i+1}^n \sum_{k=j+1}^n \varepsilon(\gcd(a[i], a[j], a[k])) \\ &= \sum_{i=1}^n \sum_{j=i+1}^n \sum_{k=j+1}^n \sum_{d|\gcd(a[i], a[j], a[k])} \mu(d) \\ &= \sum_{d=1}^{MAXV} \mu(d) \sum_{i=1}^n \sum_{j=i+1}^n \sum_{k=j+1}^n [d|\gcd(a[i], a[j], a[k])] \end{aligned}$$

Now, $d|\gcd(a[i], a[j], a[k])$ implies d separately divides each of them.

So, $\sum_{i=1}^n \sum_{j=i+1}^n \sum_{k=j+1}^n [d|\gcd(a[i], a[j], a[k])]$ is basically the number of triplets whose elements are divisible by d . If $\text{cnt}[d]$ is the number of elements in the array a divisible by d , then number of such triplets will be $\binom{\text{cnt}[d]}{3}$. Calculating $\text{cnt}[d]$ array is pretty easy: if $\text{freq}[x]$ is the number of occurrences of x , then $\text{cnt}[d] = \sum_{d|x} \text{freq}[x]$. Since we will be iterating over multiples for each d , it will take $O(n \log n)$ time. Hence we get the final answer:

$$\sum_{d=1}^{MAXV} \mu(d) \binom{\text{cnt}[d]}{3}$$

Problem 8. (CF Mike and Foam) You are given an array a and q queries. ($|a|, q \leq 2 \times 10^5$, $a[i] \leq 5 \times 10^6$). You also have a list L , which is initially empty. In each query, you will be given an integer x . If $x \in L$, then you will remove x from L , otherwise add x to L . Then you need to output the number of pairs (i, j) where $i < j$ and $i, j \in L$ such that $\gcd(a_i, a_j) = 1$.

Solution. Exercise. Check my code if needed [here](#).

Problem 9. (CF The Holmes Children) Let $f(n)$ be the number of distinct positive integer pairs (x, y) such that $x + y = n$ and $\gcd(x, y) = 1$ (additionally, $f(1) = 1$). Let $g(n) = \sum_{d|n} f(d)$. Given n and k , calculate $F_k(n)$, which is defined as:

$$F_k(n) = \begin{cases} f(g(n)) & \text{if } k = 1 \\ g(F_{k-1}(n)) & \text{if } k > 1 \text{ and } k \bmod 2 = 0 \\ f(F_{k-1}(n)) & \text{if } k > 1 \text{ and } k \bmod 2 = 1 \end{cases}$$

Solution. Exercise.

$$\begin{aligned} \sum_{i=1}^n \sum_{j=1}^{\min(m, d)} [\gcd(i, j) = 1] &= \sum_{i=1}^m \sum_{j=1}^i [\gcd(i, j) = 1] + \sum_{i=m+1}^n \sum_{j=1}^m [\gcd(i, j) = 1] \\ &= \sum_{i=1}^m \phi(i) + \sum_{i=m+1}^n \sum_{j=1}^m \sum_{d|\gcd(i, j)} \mu(d) \\ &= \sum_{i=1}^m \phi(i) + \sum_{i=m+1}^n \sum_{j=1}^m \sum_{d=1}^m \mu(d) \\ &= \sum_{i=1}^m \phi(i) + \sum_{i=m+1}^n \sum_{j=1}^m \sum_{d=1}^m [d \mid i][d \mid j] \mu(d) \\ &= \sum_{i=1}^m \phi(i) + \sum_{d=1}^m \mu(d) \sum_{i=m+1}^n [d \mid i] \sum_{j=1}^m [d \mid j] \\ &= \sum_{i=1}^m \phi(i) + \sum_{d=1}^m \mu(d) \left(\left\lfloor \frac{n}{d} \right\rfloor - \left\lfloor \frac{m}{d} \right\rfloor \right) \left(\left\lfloor \frac{m}{d} \right\rfloor \right) \end{aligned}$$

Now you can calculate it with a simple loop in $O(m)$.

Problem 4. Given n , calculate $\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j)$.

Solution. From **theorem 6**, we know that $\sum_{d|n} \phi(d) = n$. So,

$$\begin{aligned} \sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) &= \sum_{i=1}^n \sum_{j=1}^n \sum_{d|\gcd(i, j)} \phi(d) \\ &= \sum_{i=1}^n \sum_{j=1}^n \sum_{d=1}^n [d \mid i][d \mid j] \phi(d) \\ &= \sum_{d=1}^n \phi(d) \left(\sum_{i=1}^n [d \mid i] \right) \left(\sum_{j=1}^n [d \mid j] \right) \\ &= \sum_{d=1}^n \phi(d) \left(\left\lfloor \frac{n}{d} \right\rfloor \right)^2 \end{aligned}$$

which can be calculated in $O(\sqrt{n})$ for each n after the sieve (which would take $O(n \log n)$ or $O(n)$ if you use linear sieve).

Problem 5. (Hyperrectangle GCD) Let there be a k -dimensional hyperrectangle with dimensions $n_1, n_2, \dots, n_k$. Each unit cell of the hyperrectangle is addressed by $(p_1, p_2, \dots, p_k)$ where $1 \leq p_i \leq n_i$ for all $i \leq k$. The value of a cell is the gcd of its coordinates. Calculate the sum of value over all cells of the hyperrectangle.

Solution. We are basically asked to calculate the value of:

$$\sum_{x_1=1}^{n_1} \sum_{x_2=1}^{n_2} \dots \sum_{x_k=1}^{n_k} \gcd(x_1, x_2, \dots, x_k)$$

9

References

- [Multiplicative Function, Wikipedia](#)
- [Beginning Number Theory, Neville Robbins](#)
- [\[Tutorial\] Math note — Möbius inversion](#)

A000079	Powers of 2: a(n) = 2^n. (Formerly M1129 N0432)	3328
1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536, 131072, 262144, 524288, 1048576, 2097152, 4194304, 8388608, 16777216, 33554432, 67108864, 134217728, 268435456, 536870912, 1073741824, 2147483648, 4294967296, 8589934592		
A000110	Bell or exponential numbers: number of ways to partition a set of n labeled elements. (Formerly M1484 N0505)	1371
1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975, 678570, 4213597, 27644437, 190899322, 1382958545, 10480142147, 82864869804, 682676806159, 5832742285857, 51724158235372, 474869816156751, 4506715738447323, 4415208555084346, 445958869294805289, 463859833229999353, 49631246523618756274		
A000040	The prime numbers. (Formerly M0632 N0241)	1197
2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271		
A000010	Euler totient function phi(n): count numbers <= n prime to n. (Formerly M0299 N0111)	4245
1, 1, 2, 2, 4, 2, 6, 4, 6, 4, 10, 4, 12, 6, 8, 8, 16, 6, 18, 8, 12, 10, 22, 8, 20, 12, 18, 12, 20, 8, 30, 16, 20, 16, 24, 12, 36, 18, 24, 16, 40, 12, 42, 20, 24, 22, 46, 16, 42, 20, 32, 24, 52, 18, 40, 24, 36, 28, 58, 16, 60, 30, 36, 32, 48, 20, 66, 32, 44		
A000142	Factorial numbers: n! = 1*2*3*4*...*n (order of symmetric group S_n, number of permutations of n letters). (Formerly M1675 N0659)	2944
1, 1, 2, 2, 6, 24, 120, 720, 5040, 40320, 362880, 3628800, 39916800, 479001600, 6227020800, 87178291200, 1307674368000, 2952278880000, 355687428096000, 6402373705728000, 121645100408832000, 2432902080176640000, 51098942171709440000, 112408072777607680000		
A000108	Catalan numbers: C(n) = binomial(2n,n)/(n+1) = (2n)!/(n!(n+1)!). (Formerly M1459 N0577)	4309
1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694045, 35357670, 129644790, 477638700, 1767263190, 6564120420, 24460267020, 91482563640, 343859613650, 1289984147324, 4861946401452, 18363753072152, 69533550910004, 263747951750360, 1002242216651368, 3814986502092304		
A000984	Central binomial coefficients: binomial(2*n,n) = (2*n)!/(n!)^2. (Formerly M1645 N0643)	1128
1, 2, 6, 20, 70, 252, 924, 3432, 12870, 48620, 184756, 705432, 2704156, 10400600, 40116600, 155117520, 601080390, 2333606220, 9075135300, 35345263000, 137846520820, 530237874440, 2104098963720, 8233430727600, 32247603681000, 126410606437752, 495918532948104, 1946939425648112		
A001175	Pisano periods (or Pisano numbers): period of Fibonacci numbers mod n. (Formerly M2710 N1087)	177
1, 3, 8, 6, 20, 24, 16, 12, 24, 60, 10, 24, 28, 48, 40, 24, 36, 24, 18, 60, 16, 30, 48, 24, 100, 84, 72, 48, 14, 120, 30, 48, 40, 36, 00, 24, 76, 18, 56, 60, 40, 48, 08, 30, 120, 48, 32, 24, 112, 300, 72, 84, 108, 72, 20, 48, 72, 42, 58, 120, 60, 30, 48, 96, 140, 120, 136		
A008277	Triangle of Stirling numbers of the second kind, S2(n,k), n >= 1, 1 <= k <= n. (Formerly M1459 N0577)	445
1, 1, 1, 1, 3, 1, 1, 7, 6, 1, 1, 15, 25, 10, 1, 1, 31, 90, 65, 15, 1, 1, 63, 301, 350, 140, 21, 1, 1, 127, 966, 1701, 1050, 266, 28, 1, 1, 255, 3025, 7770, 6951, 2646, 462, 36, 1, 1, 511, 9330, 34105, 42525, 22827, 5880, 750, 45, 1, 1, 1023, 28501, 145750, 246730, 179407, 63907, 11880, 1155, 55, 1 (Use table graph refs latex history: raw internal format)		
A000166	Subfactorial or rencontres numbers, or derangements: number of permutations of n elements with no fixed points. (Formerly M1937 N0760)	145
1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496, 1334961, 14684570, 176214841, 2290792932, 32071101049, 481066515734, 7697064251745, 130850092279664, 2355301661033953, 44750731559645106, 895014631192902121, 18795307255050944540, 4134967596112079881, 9510425471055777937262		
A000041	a(n) is the number of partitions of n (the partition numbers). (Formerly M0663 N0244)	3096
1, 1, 2, 3, 5, 7, 11, 15, 22, 30, 42, 56, 77, 101, 135, 176, 231, 297, 385, 490, 627, 792, 1002, 1255, 1575, 1958, 2436, 3010, 3718, 4565, 5604, 6842, 8349, 10143, 12310, 14803, 17977, 21637, 26015, 31105, 37330, 44583, 53174, 63261, 75175, 89134, 105558, 124754, 147273, 173525		

$$!n = n! \sum_{i=0}^n \frac{(-1)^i}{i!}$$

A tutorial of reflection principle in combinatorics

By [firefly](#), [history](#), 11 months ago.

Introduction

A classical problem reads:

Count the paths from $(0,0)$ to $(m,n)(n > m > 0)$ that doesn't go through $y = x$ (the path can touch the line). One can only move up or right by 1 in one move.

The problem can also be described as:

Count binary strings that consist of m zeros and n ones $(n \geq m)$, such that in every prefix of the string, the number of zeros doesn't exceed the number of ones.

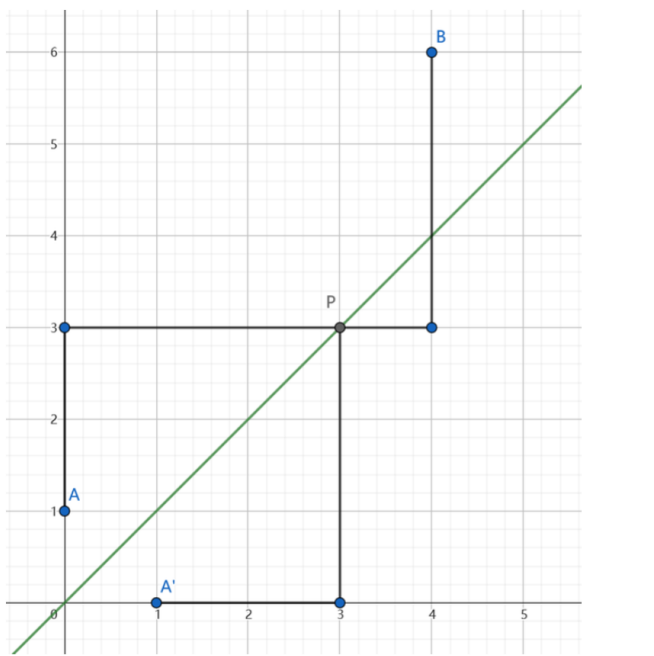
Consider how to count paths from $(0,0)$ to (m,n) without the intersection restriction. To get to (m,n) , we must move up n times and move right m times. In other words, we need to permute n right and m up, whose quantity is

$$\frac{(m+n)!}{m!n!} = \binom{m+n}{m}$$

To find the answer, we can try to count the paths that go through $y = x$, and this is when the reflection principle is utilized. The principle says:

Given point A, B at the same side of line l , the number of paths from A to B that intersect with l is equal to that of paths from A' to B , where A' is the reflection point through l .

To show why it is correct, we should first note that a path from A' to B must intersect with $y = x$. Say at point P the path first intersect with $y = x$, we can reflect the path from A' to P through l and get a path from A to B , and vice versa. This indicates that the intersecting paths from A to B and the paths from A' and B are one-to-one corresponded, and the quantity of the two are the same.



Back to our problem. Going through $y = x$ is the same as intersecting with $y = x - 1$ in this case, so $(0,0)$ can be reflected to $(1,-1)$. and the number of intersecting path is $\binom{m+n}{m-1}$. Therefore, the answer to the problem is

$$\binom{m+n}{m} - \binom{m+n}{m-1} = \frac{n-m+1}{n+1} \binom{m+n}{m}$$

In particular, when $n = m$, the formula becomes

$$\frac{1}{n+1} \binom{2n}{n}$$

which is the **Catalan number**.

Note

Note that there are two ways to generate valid paths in the first description when $n = m$, one keeps $x \geq y$ and the other keeps $x \leq y$, so the answer is $\frac{2}{n+1} \binom{2n}{n}$.

Sprague-Grundy theorem

Let's consider a state v of a two-player impartial game and let v_i be the states reachable from it (where $i \in \{1, 2, \dots, k\}, k \geq 0$). To this state, we can assign a fully equivalent game of Nim with one pile of size x . The number x is called the Grundy value or nim-value of state v .

Moreover, this number can be found in the following recursive way:

$$x = \text{mex} \{x_1, \dots, x_k\},$$

where x_i is the Grundy value for state v_i and the function mex (*minimum excludant*) is the smallest non-negative integer not found in the given set.

Viewing the game as a graph, we can gradually calculate the Grundy values starting from vertices without outgoing edges. Grundy value being equal to zero means a state is losing.

Chicken McNugget Theorem

The **Chicken McNugget Theorem** (or **Postage Stamp Problem** or **Frobenius Coin Problem**) states that for any two **relatively prime positive integers** m, n , the greatest integer that cannot be written in the form $am + bn$ for **nonnegative** integers a, b is $mn - m - n$.

A consequence of the theorem is that there are exactly $\frac{(m-1)(n-1)}{2}$ positive integers which cannot be expressed in the form $am + bn$. The proof is based on the fact that in each pair of the form $(k, mn - m - n - k)$, exactly one element is expressible.

2. Distinct values of $\lfloor \frac{n}{i} \rfloor$

Now, let's get to even more interesting facts:

Fact 2

Over all positive integers i , there are $O(\sqrt{n})$ distinct values of $\lfloor \frac{n}{i} \rfloor$.

We will discuss why this is true and how to retrieve the exact values over the exact ranges in $O(\sqrt{n})$.

Proof of Fact 2

We note that for $i > \sqrt{n}$, we have that $\lfloor \frac{n}{i} \rfloor \leq \frac{n}{i} < \frac{n}{\sqrt{n}} = \sqrt{n}$ and there are obviously at most $\sqrt{n}$ values for $i \leq \sqrt{n}$, this makes a total of at most $2\sqrt{n} \in O(\sqrt{n})$.

Now, note that these values are non-increasing as i increases. For example, for $n = 10$, we get the values (starting from $i = 1$) 10, 5, 3, 2, 2, 1, 1, 1, 1, 0, 0, ... So, to find these values, we will prove the following fact:

Fact 3

Let $S = \left\{ \lfloor \frac{n}{i} \rfloor : i \in \mathbb{Z}^+ \right\}$ — in other words, the set of all distinct values of $\lfloor \frac{n}{i} \rfloor$. Let $q \in S$ where $q > 0$ and let l be the minimum positive integer such that $\lfloor \frac{n}{l} \rfloor = q$ and let r be the maximum positive integer such that $\lfloor \frac{n}{r} \rfloor = q$, then

$$r = \left\lfloor \frac{n}{\lfloor \frac{n}{l} \rfloor} \right\rfloor$$

Techniques (A)

techniques.txt	159 lines
Recursion	
Divide and conquer	
Finding interesting points in N log N	
Algorithm analysis	
Master theorem	
Amortized time complexity	
Greedy algorithm	
Scheduling	
Max contiguous subvector sum	
Invariants	
Huffman encoding	
Graph theory	
Dynamic graphs (extra book-keeping)	
Breadth first search	
Depth first search	
* Normal trees / DFS trees	
Dijkstra's algorithm	
MST: Prim's algorithm	
Bellman-Ford	
Konig's theorem and vertex cover	
Min-cost max flow	
Lovasz toggle	
Matrix tree theorem	
Maximal matching, general graphs	
Hopcroft-Karp	
Hall's marriage theorem	
Graphical sequences	
Floyd-Warshall	
Euler cycles	
Flow networks	
* Augmenting paths	
* Edmonds-Karp	
Bipartite matching	
Min. path cover	
Topological sorting	
Strongly connected components	
2-SAT	
Cut vertices, cut-edges and biconnected components	
Edge coloring	
* Trees	
Vertex coloring	
* Bipartite graphs (=> trees)	
* 3^n (special case of set cover)	
Diameter and centroid	
K'th shortest path	
Shortest cycle	
Dynamic programming	
Knapsack	
Coin change	
Longest common subsequence	
Longest increasing subsequence	
Number of paths in a dag	
Shortest path in a dag	
Dynprog over intervals	
Dynprog over subsets	
Dynprog over probabilities	
Dynprog over trees	
3^n set cover	
Divide and conquer	
Knuth optimization	
Convex hull optimizations	
RMQ (sparse table a.k.a 2^k-jumps)	
Bitonic cycle	
Log partitioning (loop over most restricted)	
Combinatorics	

Computation of binomial coefficients
Pigeon-hole principle
Inclusion/exclusion
Catalan number
Pick's theorem
Number theory
Integer parts
Divisibility
Euclidean algorithm
Modular arithmetic
* Modular multiplication
* Modular inverses
* Modular exponentiation by squaring
Chinese remainder theorem
Fermat's little theorem
Euler's theorem
Phi function
Frobenius number
Quadratic reciprocity
Pollard-Rho
Miller-Rabin
Hensel lifting
Vieta root jumping
Game theory
Combinatorial games
Game trees
Mini-max
Nim
Games on graphs
Games on graphs with loops
Grundy numbers
Bipartite games without repetition
General games without repetition
Alpha-beta pruning
Probability theory
Optimization
Binary search
Ternary search
Unimodality and convex functions
Binary search on derivative
Numerical methods
Numeric integration
Newton's method
Root-finding with binary/ternary search
Golden section search
Matrices
Gaussian elimination
Exponentiation by squaring
Sorting
Radix sort
Geometry
Coordinates and vectors
* Cross product
* Scalar product
Convex hull
Polygon cut
Closest pair
Coordinate-compression
Quadtrees
KD-trees
All segment-segment intersection
Sweeping
Discretization (convert to events and sweep)
Angle sweeping
Line sweeping
Discrete second derivatives
Strings
Longest common substring
Palindrome subsequences

Knuth-Morris-Pratt
Tries
Rolling polynomial hashes
Suffix array
Suffix tree
Aho-Corasick
Manacher's algorithm
Letter position lists
Combinatorial search
Meet in the middle
Brute-force with pruning
Best-first (A*)
Bidirectional search
Iterative deepening DFS / A*
Data structures
LCA (2^k-jumps in trees in general)
Pull/push-technique on trees
Heavy-light decomposition
Centroid decomposition
Lazy propagation
Self-balancing trees
Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
Monotone queues / monotone stacks / sliding queues
Sliding queue using 2 stacks
Persistent segment tree